# IAsLab ORCHID: Plataforma de Orquestación y Gobernanza de Cargas de IA/ML en la Infraestructura de la Universidad Icesi

Proyecto de Grado

Presentado como parte de los requisitos para obtener los títulos de
Ingeniero Telemático e Ingeniero de Sistemas

por

Juan José De La Pava Giraldo
Ingeniería Telemática e Ingeniería de Sistemas

Juan Camilo Melo López
Ingeniería Telemática

Juan David Pacheco Vargas
Ingeniería Telemática

Tutor: Kevin David Rodríguez Belalcázar
Profesor acompañante: Mgtr. Alejandro Muñoz Bravo

Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas

Universidad Icesi
Santiago de Cali
2026

# Índice general

**Resumen** v

**Abstract** vi

**Lista de acrónimos** vii

**Glosario de términos** viii

**1 Motivación y antecedentes** 1
- 1.1 Contexto . . . 1
- 1.2 Antecedentes del problema . . . 2
- 1.3 Justificación . . . 3

**2 Descripción del problema** 5
- 2.1 Identificación del problema . . . 5
- 2.2 Formulación del problema . . . 6

**3 Objetivos** 7
- 3.1 Objetivo general . . . 7
- 3.2 Objetivos específicos . . . 7

**4 Marco teórico** 8
- 4.1 El ciclo de vida de las cargas de inteligencia artificial . . . 8
- 4.2 Despliegue e inferencia de modelos . . . 9
- 4.3 Contenedores y orquestación de cargas de cómputo . . . 10
- 4.4 Extensibilidad declarativa del orquestador . . . 11
- 4.5 Gobernanza de recursos compartidos . . . 12
- 4.6 Identidad federada y control de acceso . . . 13
- 4.7 Observabilidad y telemetría de infraestructura acelerada . . . 14
- 4.8 Marco conceptual y categorías de análisis . . . 16

**5 Estado del arte** 18
- 5.1 Estrategia de revisión documental . . . 18
- 5.2 Gestión y planificación en clústeres de aceleradores a gran escala . . . 18

5.3 Infraestructuras de aceleración en entornos universitarios . . . 19
5.4 Ecosistemas de MLOps y plataformas de ciclo de vida . . . 20
5.5 Pasarelas de inferencia y gobernanza de interfaces de programación . . . 21
5.6 Motores de inferencia como componentes de la plataforma . . . 22
5.7 Síntesis comparativa y vacíos identificados . . . 23

**6 Metodología** **25**
6.1 Estrategia metodológica . . . 25
6.2 Elicitación y especificación de requerimientos . . . 26
6.3 Riesgos y limitaciones del proyecto . . . 26

**7 Fases de desarrollo y cronograma** **27**
7.1 Fases de desarrollo del proyecto . . . 27
7.2 Cronograma . . . 28

**8 Contribución y resultados del proyecto de grado** **30**
8.1 Aportes relacionados con el objeto del proyecto . . . 30
8.2 Aportes relacionados con el desarrollo de capacidades del investigador . . . 30
8.3 Resultados y entregables . . . 31

**9 Avances del proyecto** **33**
9.1 Primera etapa: cimientos del clúster . . . 33
9.2 Segunda etapa: servicio de modelos y gobierno . . . 34
9.3 Mediciones preliminares . . . 36
9.4 Reuniones de seguimiento . . . 36
9.5 Estado al cierre de la etapa . . . 37

**A Análisis de participación** **44**
A.1 Identificación de los interesados . . . 44
A.2 Clasificación en la matriz poder-interés . . . 45
A.3 Análisis ético y de impacto . . . 46

**B Árbol de problemas** **48**

**C Árbol de objetivos** **50**

**D Requerimientos del sistema por objetivo** **52**
D.1 Módulo de observabilidad (Objetivo 1) . . . 52
D.2 Gobernanza de recursos y cuotas por rol (Objetivo 2) . . . 54
D.3 Despliegue de modelos y ejecución de cargas de IA (Objetivo 3) . . . 56
D.4 Evaluación de desempeño (Objetivo 4) . . . 58

**E Análisis de riesgos y limitaciones** **59**

# Índice de tablas

4.1 Definición operativa de los conceptos del marco teórico y objetivo específico que los emplea. . . . . 16

5.1 Comparación de alternativas previas frente a los requerimientos de la plataforma . . 23

7.1 Fases de desarrollo del proyecto, su correspondencia con los objetivos específicos y su duración. . . . . 27

8.1 Trazabilidad entre objetivos específicos, resultados esperados y entregables. . . . . 31

9.1 Mediciones preliminares del POC (solo CPU; no son una evaluación de desempeño). 36

A.1 Interesados del proyecto, su categoría y su relación con la plataforma. . . . . 44
A.2 Poder, interés y estrategia asignada a cada interesado. . . . . 45

# Índice de figuras

4.1 Fases del ciclo de vida de un modelo y alcance del proyecto . . . . . 9
4.2 Bucle de reconciliación de un controlador de Kubernetes . . . . . 11
4.3 Identidad federada y control de acceso . . . . . 14
4.4 Flujo de la telemetría en el clúster . . . . . 16

7.1 Diagrama de Gantt del proyecto . . . . . 29

9.1 Flujo de un despliegue en el POC . . . . . 35
9.2 Arquitectura del POC al cierre de la primera etapa . . . . . 38

A.1 Matriz poder-interés . . . . . 46

B.1 Árbol de problemas . . . . . 49

C.1 Árbol de objetivos . . . . . 51

# Resumen

**Palabras clave**:

# Abstract

**Keywords**:

# Lista de acrónimos

**API** Interfaz de programación de aplicaciones (*Application Programming Interface*)

**CNI** Interfaz de red de contenedores (*Container Network Interface*)

**CPU** Unidad central de procesamiento (*Central Processing Unit*)

**CUDA** Plataforma de cómputo paralelo de NVIDIA (*Compute Unified Device Architecture*)

**DCGM** Gestor de GPU para centros de datos, de NVIDIA (*Data Center GPU Manager*)

**E/S** Entrada y salida

**GGUF** Formato de archivo para modelos cuantizados empleado por *llama.cpp* y Ollama

**GPU** Unidad de procesamiento gráfico (*Graphics Processing Unit*)

**HPC** Computación de alto rendimiento (*High Performance Computing*)

**HTTP** Protocolo de transferencia de hipertexto (*Hypertext Transfer Protocol*)

**IA** Inteligencia artificial

**IAsLab** Laboratorio de Inteligencia Artificial y Arquitectura de Software de la Universidad Icesi

**KV** Clave-valor (*Key-Value*)

**LLM** Modelo de lenguaje de gran escala (*Large Language Model*)

**ML** Aprendizaje automático (*Machine Learning*)

**MLOps** Operaciones de aprendizaje automático (*Machine Learning Operations*)

**OAuth** Protocolo abierto de autorización (*Open Authorization*)

**OIDC** Capa de identidad sobre OAuth 2.0 (*OpenID Connect*)

**PCIe** Interconexión exprés de componentes periféricos (*Peripheral Component Interconnect Express*)

**POC** Prueba de concepto (*Proof of Concept*)

**RAM** Memoria de acceso aleatorio (*Random Access Memory*)

**RBAC** Control de acceso basado en roles (*Role-Based Access Control*)

**SSH** Protocolo de acceso remoto seguro (*Secure Shell*)

**VPN** Red privada virtual (*Virtual Private Network*)

**VRAM** Memoria de acceso aleatorio de video (*Video Random Access Memory*)

# Glosario de términos

**SAAMFI** Servicio institucional de identidad de la Universidad Icesi. Funciona exclusivamente como proveedor de identidad (*Identity Provider*), en el mismo sentido que Keycloak o Auth0: registra a los usuarios y su esquema de roles y permisos, y emite un *token* que cada sistema integrado consume según sus propias reglas. No impone políticas de autorización sobre los sistemas que lo usan, de modo que la traducción de un rol en un límite de consumo de *hardware* corresponde a la plataforma desarrollada en este proyecto.

**TRAINI** Plataforma desarrollada por un proyecto de grado anterior del IAsLab para configurar, ejecutar y monitorear tareas de entrenamiento distribuido de modelos de inteligencia artificial. Centraliza la configuración, ejecución y seguimiento de los entrenamientos, aprovisiona los nodos de procesamiento e integra SAAMFI como proveedor institucional de identidad.

# 1. Motivación y antecedentes

## 1.1. Contexto

Llevar un modelo de aprendizaje automático a producción exige más que entrenarlo. Sculley et al. (2015) señalan que su código es solo una pequeña fracción de un sistema real, rodeado de una infraestructura extensa que incluye el servicio y el monitoreo de los modelos. Kreuzberger et al. (2023) definen las operaciones de aprendizaje automático (MLOps) como un paradigma que abarca la conceptualización, implementación, monitoreo, despliegue y escalabilidad de estos productos, e incluyen un componente dedicado a vigilar de forma continua el desempeño en servicio y el estado de la infraestructura. Aun así, Eken et al. (2026) advierten que un cuerpo de conocimiento integrado sobre MLOps sigue sin lograrse, porque su alcance es amplio y abarca desafíos muy diversos de esa transición.

La infraestructura mencionada exige, además, una capacidad de procesamiento creciente. Desde la llegada del aprendizaje profundo a comienzos de la década de 2010, el cómputo necesario para entrenar los sistemas más avanzados se ha duplicado aproximadamente cada seis meses (Sevilla et al., 2022). La industria domina una porción cada vez mayor de ese poder de cálculo, mientras la academia cuenta con menos del que necesita para investigar, sobre todo en unidades de procesamiento gráfico (GPU) (Ahmed et al., 2023). En ese escenario, las GPU que ya posee una universidad adquieren un valor estratégico.

Cuando ese *hardware* se comparte, optimizar su uso se vuelve un reto estructural. Incluso en plataformas empresariales multiusuario de aprendizaje profundo, algunos trabajos presentan una utilización notablemente baja de las GPU asignadas, lo que desperdicia recursos valiosos y reduce la productividad del desarrollo (Gao et al., 2024). Dicho estudio analizó 400 casos reales en un entorno corporativo de gran tamaño, elegidos entre los que aprovechaban en promedio la mitad o menos de su capacidad, e identificó en ellos 706 problemas de este tipo, originados en el código de cada trabajo por cómputo insuficiente en el acelerador o por interrupciones de tareas ajenas a él. Si el problema aparece a esa escala, es razonable que un laboratorio académico también necesite herramientas para detectarlo y controlarlo.

Ese reto se presenta, en una dimensión más modesta, en los equipos de cómputo que el Laboratorio de Inteligencia Artificial y Arquitectura de Software (IAsLab) de la Universidad Icesi destina a la ciencia de datos y la inteligencia artificial. El macroproyecto *Ecosistema computacional de servicios de software para el desarrollo de proyectos del IAsLab para Industria 4.0 y e-Salud en el contexto de la Transformación Digital*, una iniciativa de la universidad, tiene como propósito reducir la brecha entre la investigación aplicada del laboratorio y su adopción en entornos productivos. Para ello, el macroproyecto busca consolidar una base común de componentes y servicios reutilizables y estandarizados que eleven la madurez tecnológica de sus desarrollos y faciliten su paso de la experimentación académica a soluciones robustas y seguras, acordes con las exigencias

de los sectores industrial y clínico.

**[TODO: El proyecto TRAINI ahora se denomina TrainLab. Actualicen el nombre en todo el documento, incluidos el glosario, los capítulos 1 y 4 (también la Figura 4.1) y los anexos.]**

En ese marco, un proyecto de grado anterior construyó la plataforma TRAINI, que actúa como plano de control (*control plane*) del entrenamiento distribuido. Esa herramienta centraliza la configuración, ejecución y seguimiento de los entrenamientos, aprovisiona los nodos de procesamiento e integra SAAMFI como proveedor institucional de identidad.

Alrededor de esa infraestructura intervienen actores con necesidades distintas. Los estudiantes y profesores de la universidad requieren GPU para sus proyectos e investigaciones, y se identifican mediante sus roles institucionales. Los administradores del IAsLab operan los equipos y asumen su soporte. Quienes usan TRAINI comparten esas máquinas, que además deben seguir disponibles para las clases.

## 1.2. Antecedentes del problema

Antes de TRAINI, el IAsLab carecía de una plataforma estandarizada para administrar el acceso a su infraestructura de cómputo. La habilitación de usuarios se realizaba mediante credenciales de red privada virtual (VPN, *Virtual Private Network*) y permisos otorgados a solicitud individual, lo que permitía el acceso a estudiantes de asignaturas afines como Ingeniería de Software IV, a integrantes del laboratorio y, en general, a docentes o estudiantes que tramitaran su requerimiento. No obstante, dicho trámite dependía de gestiones manuales y en ocasiones conllevaba tiempos de espera prolongados para su aprobación y configuración. La ausencia de un mecanismo automatizado de aprovisionamiento desincentivaba el aprovechamiento continuo de los recursos, por lo cual parte de la comunidad estudiantil recurría a servicios externos como Google Colab o prescindía de cómputo especializado, lo que produjo una subutilización sostenida del *hardware*. A la fecha, el laboratorio emplea sus equipos de forma experimental y todavía no en explotación continua.

**[TODO: Revisen la redacción de este párrafo. «Poner un modelo en operación» y «el laboratorio desplegaba modelos ejecutando a mano» usan un registro coloquial y repiten la idea. Reformúlenlo en un lenguaje técnico y más preciso, por ejemplo con el término «despliegue manual» definido una sola vez y usado de forma consistente.]**

Poner un modelo en operación también era un proceso manual. Antes de este proyecto, el laboratorio desplegaba modelos ejecutando a mano *scripts* de Python que instalaban los motores de inferencia y sus dependencias directamente sobre el sistema operativo de las estaciones. En ausencia de una plataforma o interfaz de autoservicio que abstrajera la infraestructura subyacente, este procedimiento no podía gestionarse de forma desacoplada del anfitrión. La puesta en servicio exigía operar directamente sobre los nodos con credenciales de acceso a las máquinas, otorgadas vía VPN y permisos administrativos, condición técnica que supeditaba el despliegue a contar previamente con dicho acceso. TRAINI automatizó el aprovisionamiento de nodos para el entrenamiento distribuido, pero no abordó las etapas posteriores al entrenamiento. Hoy ninguna plataforma del laboratorio

permite desplegar y servir modelos ya entrenados ni gobernar el uso compartido de las GPU, y los administradores no pueden observar en tiempo real el consumo exacto de la infraestructura.

Las universidades que comparten clústeres de GPU enfrentan problemas semejantes. Xu et al. (2025), a partir de la operación de un clúster de más de 160 GPU al servicio de más de 480 usuarios de su universidad, señalan que compartir estos recursos es esencial para aprovecharlos y ampliar el acceso, aunque gestionarlos plantea desafíos que van desde la configuración del sistema hasta el reparto equitativo entre usuarios, con personal limitado para operarlos. Según los autores, muchas universidades adoptan una herramienta conocida con configuraciones subóptimas y terminan subutilizando recursos ya limitados. Un esquema simple y común, que entrega a los usuarios acceso directo por consola a las máquinas con GPU, carece de mecanismos de aislamiento y de gestión, obliga a cada usuario a configurar su entorno y a coordinar a mano el uso de las GPU, y dificulta que el reparto sea equitativo.

En una plataforma académica de mayor escala, un mecanismo de gobernanza tuvo efectos medibles. En la National Research Platform, un clúster Kubernetes multiusuario que en 2024 atendió a investigadores de 50 campus, la introducción de un sistema de reservas para las GPU de mayor demanda elevó la utilización promedio de las GPU del clúster del 21.49 % al 31.37 %, aunque sus operadores reconocen que aún hay margen de mejora (Weitzel et al., 2025). Estos antecedentes coinciden con la situación del IAsLab, donde el acceso directo y manual a los equipos no ofrece un reparto controlado de las GPU ni visibilidad sobre su uso, condiciones que exponen la infraestructura a un riesgo de monopolización a medida que crece el número de usuarios.

## 1.3. Justificación

Este proyecto responde a una necesidad compartida por el IAsLab, el Departamento de Computación y Sistemas Inteligentes y la universidad. El laboratorio despliega hoy sus modelos de forma manual, no cuenta con reglas que repartan sus GPU entre los usuarios y no dispone de una vista del consumo de su infraestructura, mientras crece la comunidad que podría aprovecharla. Una solución que automatice el despliegue, gobierne el acceso mediante cupos y observe el estado del *hardware* sustituye la entrega manual de credenciales por un servicio al que acceden estudiantes y profesores de la universidad. Con esa capacidad, la plataforma puede sostener proyectos de otras áreas, como los de Industria 4.0 y e-Salud.

Ese cambio amplía el acceso al cómputo especializado. Al sustituir un esquema de asignación manual y tiempos de espera prolongados por un servicio con reglas de uso explícitas, la capacidad instalada queda al alcance de más estudiantes y profesores, que dejan de depender de suscripciones comerciales que no todos pueden costear, y se fortalecen otros proyectos que requieren esos recursos. La universidad, por su parte, obtiene un mayor retorno de la inversión que ya hizo en equipos, al usarlos en lugar de adquirir créditos en nubes públicas.

Ese mejor aprovechamiento tiene también un efecto ambiental. En un clúster universitario compartido, el consumo base de los equipos, que incluye la potencia en reposo de sus componentes, se mantiene relativamente constante sin importar cuántas GPU estén ocupadas, y la GPU representa la mayor parte de la potencia demandada (Xu et al., 2025). Un acelerador asignado que no se usa

gasta energía sin producir resultados, de modo que gobernar su reparto y detectar la subutilización favorece un uso más eficiente de la energía del laboratorio.

Por último, el proyecto es relevante porque el escenario en el que se desarrolla reúne características que lo distinguen de los entornos industriales de gran escala. Un laboratorio universitario con pocos nodos de GPU usa las mismas máquinas para las clases y para la investigación, y necesita gobernar el servicio de modelos y observar el estado del *hardware* sobre infraestructura propia. Documentar el diseño, la operación y la evaluación de una plataforma en esas condiciones genera una experiencia transferible a otras instituciones académicas con recursos limitados.

# 2. Descripción del problema

## 2.1. Identificación del problema

La infraestructura de procesamiento gráfico (GPU) del IAsLab presenta limitaciones estructurales para operar como un entorno compartido, regulado y observable al servicio de la comunidad académica. Esta situación se manifiesta en tres síntomas. En primer lugar, la puesta en operación de modelos de inteligencia artificial se realiza mediante procedimientos manuales directamente sobre las máquinas físicas. En segundo lugar, la asignación de recursos y la habilitación de usuarios dependen de solicitudes individuales gestionadas de forma manual y con tiempos de espera prolongados, sin mecanismos de autoservicio. En tercer lugar, la supervisión de la infraestructura carece de telemetría en tiempo real sobre el consumo y el estado de los nodos.

El origen de esta situación radica en la ausencia de una capa de *software* de orquestación y gestión que administre la etapa de inferencia sobre la infraestructura de cómputo del laboratorio. De esta causa principal se desprenden tres factores causales.

- La carencia de un plano de servicio que gestione motores de inferencia obliga a ejecutar dependencias y *scripts* de despliegue de forma manual en el sistema operativo anfitrión.
- La falta de integración entre el proveedor institucional de identidad (SAAMFI) y un motor de políticas de cuotas impide traducir los roles de los usuarios en cupos de cómputo, lo que perpetúa la asignación manual de accesos sin reglas explícitas.
- La inexistencia de un sistema centralizado de observabilidad priva a los administradores de visibilidad en tiempo real sobre el estado de los nodos y la carga de inferencia que atienden.

La consecuencia primordial de estas deficiencias es la subutilización sostenida de una infraestructura en la que la universidad ya invirtió, junto con la interrupción del ciclo de vida de los modelos tras la etapa de entrenamiento. Dicha subutilización resulta crítica al contrastarse con el volumen de la demanda potencial que representan los programas académicos de tecnologías de la información de la institución. De acuerdo con las estadísticas oficiales de matrícula de la Universidad Icesi, la comunidad directamente vinculada a estas áreas comprende 667 estudiantes de pregrado y 190 estudiantes de posgrado en programas avanzados de computación e inteligencia artificial. En este último nivel, la demanda especializada se concentra en la Maestría en Inteligencia Artificial (89 estudiantes), la Maestría en Ciencia de Datos (74 estudiantes), el Doctorado en Ingeniería (13 estudiantes), la Especialización en Ingeniería de Datos (8 estudiantes) y la Especialización en Inteligencia Artificial Aplicada (6 estudiantes). Frente a una comunidad de más de 850 estudiantes con requerimientos formativos y proyectos aplicados en analítica y aprendizaje automático, la ausencia de una vía de autoservicio y gobernanza restringe el aprovechamiento de los aceleradores e impide que la capacidad instalada absorba la demanda académica.

Los efectos secundarios se manifiestan de manera diferenciada en la comunidad y en la operación técnica. Para los estudiantes y profesores, la dificultad de desplegar modelos y la demora en los trámites de acceso limitan la validación práctica de sus investigaciones o los obligan a incurrir en costos de plataformas comerciales en la nube. Para los administradores, la gestión individual de credenciales y la atención particular de cada solicitud generan una sobrecarga operativa recurrente. Sobre la infraestructura compartida, además, la ausencia de cupos sujetos a normas y de monitoreo continuo crea riesgos latentes de saturación desordenada de las GPU y de fallas operativas no detectadas, tales como desbordamientos de memoria en los aceleradores, que pueden degradar la disponibilidad de los nodos para las clases y los proyectos concurrentes. El Anexo B presenta de forma gráfica esta jerarquía de causas y efectos.

## 2.2. Formulación del problema

La infraestructura de cómputo del IAsLab carece de mecanismos para operar como un entorno multiusuario compartido, gobernado y observable, debido a que la puesta en servicio de modelos, la asignación de recursos y la supervisión del *hardware* se gestionan mediante procedimientos manuales desacoplados entre sí. Como consecuencia, la capacidad instalada se subutiliza, los usuarios encuentran demoras para acceder a cómputo especializado, el ciclo de vida de los modelos se interrumpe tras el entrenamiento y el clúster permanece expuesto a saturaciones no reguladas o fallas inadvertidas. Al tratarse de un proyecto de desarrollo tecnológico que construye una solución de *software* propia, la problemática se sintetiza en la siguiente pregunta de ingeniería. ¿De qué manera diseñar y construir una plataforma sobre la infraestructura del IAsLab que permita desplegar modelos de inferencia, regular el uso de las GPU mediante cupos asignados según las reglas de reparto asociadas a los roles institucionales y supervisar el estado del *hardware*, garantizando la estabilidad y disponibilidad de los recursos compartidos? Resolver esta pregunta permite sustituir la administración manual por un servicio integrado que articule el despliegue, la gobernanza y la observabilidad en beneficio de la docencia y la investigación.

# 3. Objetivos

## 3.1. Objetivo general

**[TODO: El objetivo general contiene dos verbos, «construir» la plataforma y «consolidar» el ciclo de vida de MLOps. Definan cuál es el propósito principal del proyecto. Si construir la plataforma es un medio para consolidar el ciclo de vida, el verbo del objetivo debería ser el segundo, y la plataforma pasaría al «mediante». Mantengan un solo verbo y verifiquen que los objetivos específicos, en conjunto, permitan alcanzarlo.]**

Construir la plataforma de orquestación y gobernanza de cargas de inteligencia artificial y aprendizaje automático del IAsLab, mediante módulos de despliegue de inferencia, gobernanza de recursos por rol y observabilidad de infraestructura, para consolidar el ciclo de vida de MLOps *on-premise* en la Universidad Icesi.

## 3.2. Objetivos específicos

1. Desarrollar un módulo de observabilidad de la plataforma para la captura y visualización de métricas de *hardware* y de carga de trabajo de los nodos de cómputo del IAsLab, orientado a la supervisión del estado de dichos nodos.
2. Implementar un sistema de cuotas y restricciones de infraestructura, acoplado a los roles provistos por SAAMFI y a un rol de administrador de la plataforma, que reserve cupos de cómputo para las cargas de inferencia, con el fin de prevenir la monopolización de las GPU.
3. Implementar sobre el clúster de cómputo del IAsLab el despliegue concurrente de modelos de inteligencia artificial ya entrenados, mediante motores de inferencia empaquetados como imágenes de contenedor, con el fin de llevar los modelos del laboratorio a una fase de servicio consumible.
4. Evaluar el desempeño de la plataforma mediante pruebas de carga con usuarios concurrentes, con el fin de determinar hasta qué nivel de concurrencia sostiene el servicio.

# 4. Marco teórico

Construir una plataforma que despliegue modelos de inteligencia artificial sobre *hardware* compartido, gobierne quién los ejecuta y observe lo que ocurre en la infraestructura mientras lo hacen exige integrar conocimiento de cinco dominios de la ingeniería de sistemas, que abarcan el ciclo de vida de los sistemas de aprendizaje automático, el servicio de modelos en producción, la orquestación de contenedores y su modelo de extensión, la gobernanza de recursos compartidos entre usuarios con derechos desiguales, y la observabilidad de infraestructura acelerada. Este capítulo los recorre de lo general a lo particular, desde la fase del ciclo de vida que el proyecto ocupa hasta las técnicas concretas que la instrumentan, y cierra con las definiciones operativas y las categorías bajo las cuales se analiza el resultado.

## 4.1. El ciclo de vida de las cargas de inteligencia artificial

MLOps designa un paradigma que reúne buenas prácticas, conceptos y una cultura de desarrollo para la conceptualización, implementación, monitoreo, despliegue y escalabilidad de extremo a extremo de los productos de aprendizaje automático, y que se apoya en tres disciplinas, el aprendizaje automático, la ingeniería de *software* (en particular DevOps) y la ingeniería de datos (Kreuzberger et al., 2023). El enfoque surgió para reducir el esfuerzo y mejorar la integración entre quienes llevan los modelos al entorno de producción (Lima et al., 2022). Sculley et al. (2015) explican por qué esa integración pesa tanto. En un sistema de aprendizaje automático real, el código del modelo ocupa una fracción pequeña del conjunto, y la infraestructura que lo rodea, que incluye la configuración, la infraestructura de servicio, la gestión de los recursos de cómputo y el monitoreo, es extensa y compleja. Los mismos autores advierten que estos sistemas tienen una capacidad especial para acumular deuda técnica, difícil de detectar porque se sitúa en el nivel del sistema completo, fuera del código del modelo.

Dentro de ese ciclo, la literatura distingue dos fases con objetivos distintos. Ye et al. (2024) describen el entrenamiento como cargas de larga duración que se ejecutan fuera de línea, para las que el planificador busca un buen desempeño de cada trabajo, una alta utilización del centro de datos y equidad entre usuarios, y la inferencia como un servicio en línea que atiende las solicitudes de los usuarios, con exigencias mayores de latencia y con peticiones en ráfagas difíciles de predecir. Por esa diferencia, los autores estudian por separado los planificadores de cada fase. La distinción se aplica a cualquier tipo de modelo. Un clasificador clásico, un modelo de visión por computador y un modelo generativo de gran escala atraviesan el mismo ciclo y presentan la misma separación entre la fase que produce el artefacto y la fase que lo pone en servicio. Lo que cambia entre ellos es el perfil de consumo y el conjunto de métricas relevantes, sin que la estructura del ciclo se altere. El proyecto se ubica por completo en la fase de servicio, para cargas de inteligencia artificial en general. El entrenamiento corresponde a TRAINI, el proyecto de grado previo del mismo macroproyecto, y queda fuera del alcance de este documento. La Figura 4.1 presenta ambas fases y la parte del ciclo

que cubre este proyecto sobre los nodos con GPU que las dos comparten.

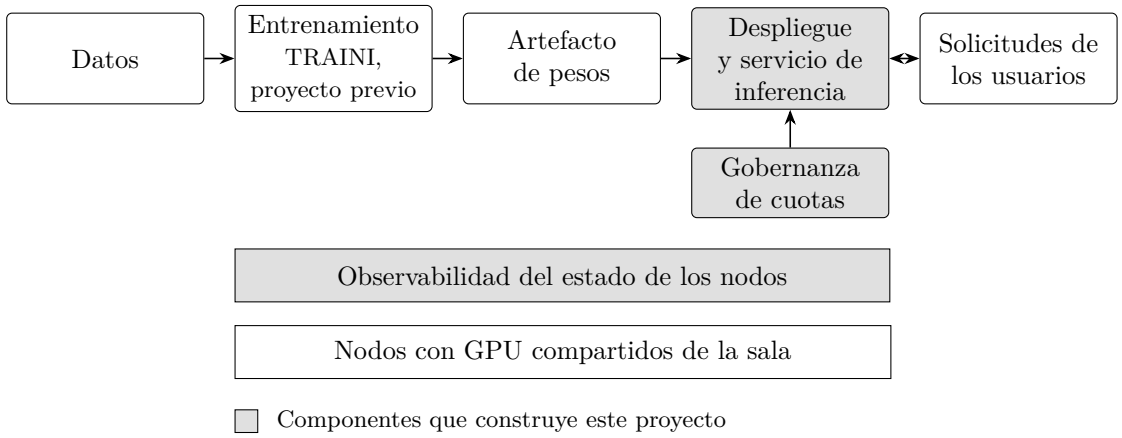



**Figura 4.1:** *Fases del ciclo de vida de un modelo y alcance del proyecto. En gris aparecen los componentes que construye este proyecto; el entrenamiento corresponde a TRAINI, el proyecto de grado previo del mismo macroproyecto, y ambas fases se ejecutan sobre los mismos nodos. Elaboración propia.*

## 4.2. Despliegue e inferencia de modelos

Desplegar un modelo consiste en instalar un artefacto ya entrenado en un entorno de ejecución donde pueda recibir solicitudes de forma sostenida, e inferir es la operación por la cual ese artefacto produce una salida a partir de una entrada nueva. Entre ambos se interpone el motor de inferencia, componente que carga los pesos en la memoria del acelerador, gestiona el conjunto de solicitudes concurrentes y expone una interfaz de red por la que los clientes consumen el modelo. El compromiso entre latencia y aprovechamiento es común a cualquier modelo, porque ubicar varias cargas de inferencia en un mismo equipo o aumentar el tamaño del lote mejora la utilización y el *throughput*, pero puede elevar la latencia (Ye et al., 2024).

En los modelos de lenguaje de gran escala (*Large Language Models*, LLM), ese compromiso es más delicado, porque la baja latencia y el alto *throughput* son objetivos complementarios pero a menudo en conflicto (Miao et al., 2025). El motor enfrenta, además, un cuello de botella particular, porque la caché de atención crece con la longitud de cada petición y, bajo solicitudes concurrentes de longitud variable, fragmenta la memoria del acelerador hasta desperdiciar una fracción sustancial de la capacidad instalada. Kwon et al. (2023) resuelven esa fragmentación paginando la caché en bloques de tamaño fijo, con una analogía directa a la memoria virtual de un sistema operativo. Yu et al. (2022) atacan el mismo problema desde la programación de peticiones y muestran que planificar a nivel de iteración, en lugar de esperar a que el lote completo termine, eleva de forma sustancial el *throughput* agregado del servidor. Miao et al. (2025) organizan estas técnicas, junto con la compresión de modelos y la programación de *kernels*, como un espectro continuo de optimización que abarca desde el algoritmo hasta el sistema. De ese compromiso se derivan las métricas con que el proyecto evalúa el servicio de lenguaje, que son el tiempo hasta el primer *token*, la latencia entre *tokens*, el *throughput* agregado y el tiempo de espera en cola.

Los modelos que no son de lenguaje reciben el mismo tratamiento. Un clasificador o un modelo de visión por computador se despliega con su propio artefacto y cuenta con marcos de servicio compatibles con Kubernetes, como TensorFlow Serving o Seldon (Kreuzberger et al., 2023). Su desempeño se mide con la latencia de petición y la tasa de solicitudes atendidas, sin depender de métricas de *tokens*. La plataforma los despliega, gobierna y observa por el mismo camino que un modelo de lenguaje, de modo que ni la gobernanza ni la telemetría dependen de la familia del modelo.

Dentro de ese espectro, la cuantización posentrenamiento ocupa un lugar particular. La técnica reduce la precisión numérica con la que se representan los pesos, con una degradación de calidad acotada y medida experimentalmente (Frantar et al., 2023; Lin et al., 2024; Dettmers et al., 2022; Chen et al., 2026). Para este proyecto su relevancia está en la capacidad que libera, porque explica por qué modelos de 7 000 a 14 000 millones de parámetros caben dentro de los 24 GB de memoria de video de un nodo con tarjeta RTX 4090, y por tanto por qué el laboratorio puede ponerlos en servicio sin recurrir a inferencia distribuida entre nodos. La plataforma despliega pesos que ya llegan cuantizados, o se apoya en la cuantización que el propio motor aplica al cargar el modelo, como hace *llama.cpp*, sin ejecutar ningún proceso propio de cuantización.

Los motores disponibles ocupan puntos distintos de ese espectro, y esa diversidad es la razón teórica por la cual la plataforma no puede acoplarse a ninguno. Miao et al. (2025) observan que las decisiones de diseño de cada sistema de servicio responden en gran medida a su objetivo de optimización prioritario, y vLLM, por ejemplo, usa la atención paginada para aumentar el tamaño del lote y, con él, el *throughput*. Otros motores privilegian la latencia de un único flujo o atienden familias de modelos que no son de lenguaje, con formatos de artefacto y perfiles de cómputo propios, y la Sección 5.6 compara los dos que el laboratorio ha considerado. La literatura revisada no ofrece una comparación controlada entre estas opciones sobre el tipo de *hardware* que el IAsLab tiene instalado, de modo que cualquier preferencia operativa del laboratorio constituye una decisión de ingeniería propia, pendiente de respaldo experimental. Esa ausencia obliga al proyecto a tratar el motor como pieza intercambiable y a producir la comparación mediante medición.

## 4.3. Contenedores y orquestación de cargas de cómputo

Un motor de inferencia, con sus pesos, sus dependencias de CUDA (*Compute Unified Device Architecture*, la plataforma de cómputo paralelo de NVIDIA) y sus versiones concretas de bibliotecas, constituye una unidad de *software* cuya reproducción por instalación manual es frágil. El contenedor la empaqueta junto con su entorno de ejecución completo en una imagen inmutable, de modo que lo que se ejecuta en un nodo es idéntico a lo que se probó en otro; el orquestador resuelve el problema siguiente, que consiste en decidir en qué máquina se ejecuta cada unidad, reiniciarla cuando falla y exponerla a la red de forma estable. Kubernetes se consolidó como el orquestador de referencia para esa tarea. Un análisis bibliométrico lo describe como el estándar de la orquestación de contenedores y ubica la programación de recursos, los microservicios y la inteligencia artificial entre sus temas de investigación más activos (Carrión, 2022), y una revisión multivocal documenta, junto a los beneficios que los profesionales reportan, la falta de herramientas de diagnóstico, es decir, de monitoreo y registro, entre los retos que enfrentan (Shamim et al., 2022). Ese balance

sostiene dos decisiones del proyecto a la vez, que son adoptar Kubernetes como sustrato en lugar de construir un programador de clúster propio, y prever desde el diseño la capa de observabilidad que cubra el vacío de diagnóstico que la propia literatura señala.

Las primitivas que el orquestador ofrece se aplican de forma directa al caso del laboratorio. Los selectores de nodo, junto con las marcas y tolerancias, dirigen las cargas que requieren aceleración a las máquinas con tarjeta gráfica y reservan las demás para los servicios de control. Los espacios de nombres dividen el clúster en fronteras lógicas de administración sobre las que se aplican límites agregados de consumo. Los volúmenes persistentes desacoplan el almacenamiento de los pesos del ciclo de vida de los contenedores que los sirven, lo que evita descargar decenas de gigabytes cada vez que un despliegue se reinicia. Estos mecanismos se aplican a cualquier tipo de carga, y esa generalidad permite gobernar con las mismas herramientas los distintos modelos en servicio y los servicios auxiliares de la plataforma.

## 4.4. Extensibilidad declarativa del orquestador

Kubernetes no fue diseñado para cargas de inteligencia artificial y sin embargo las hospeda sin que su núcleo haya sido rediseñado. El mecanismo que lo permite es su modelo declarativo, cuyo principio central describen Burns et al. (2016) a partir de una década operando los sistemas Borg, Omega y Kubernetes. Según ese principio, el usuario declara el estado que desea para el clúster, y un controlador ejecuta de forma permanente un bucle de reconciliación que compara el estado declarado con el actual y actúa hasta hacerlos converger. Como cada acción se decide a partir del estado presente, sin depender de una máquina de estados que recuerde por dónde iba el proceso, un controlador que falla y se reinicia retoma el trabajo desde lo que encuentra. La Figura 4.2 representa ese bucle, y esa propiedad es la que permite operar una infraestructura descrita íntegramente en archivos de configuración versionados y redesplegarla sin reconstruirla a mano. El proyecto la necesita porque el clúster se levanta desde cero sobre las estaciones de trabajo de la sala.

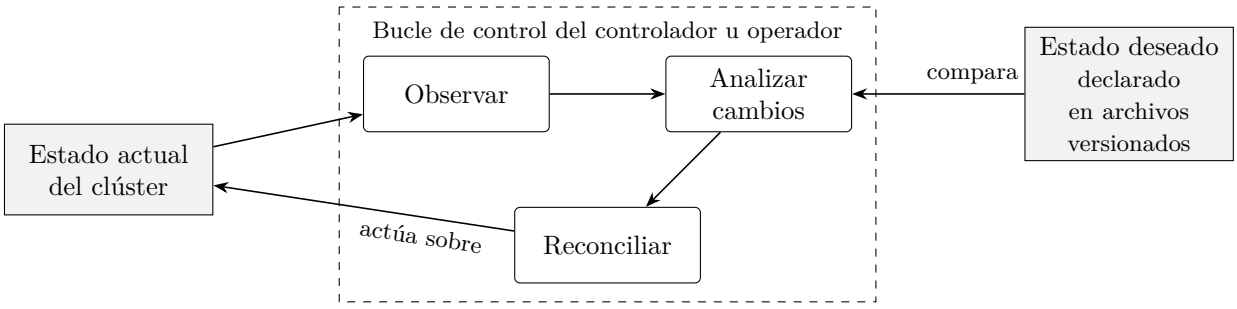



**Figura 4.2:** *Bucle de reconciliación de un controlador de Kubernetes. El controlador observa el estado actual del clúster, lo compara con el estado deseado y actúa hasta hacerlos converger. Adaptado de Xu et al. (2024) (Figura 1) y de Burns et al. (2016).*

Sobre ese modelo se apoya la vía de extensión. Una definición de recurso personalizado (*Custom Resource Definition*, CRD) registra en el servidor de la interfaz de programación de aplicaciones

(API, *Application Programming Interface*) un tipo de objeto que Kubernetes no conocía, junto con el esquema que valida sus campos, de modo que un concepto propio de un dominio pasa a ser un objeto de primera clase del clúster. El controlador que acompaña a ese tipo de recurso, y que encapsula el conocimiento operativo necesario para materializarlo, recibe el nombre de operador. Xu et al. (2024) estudiaron 210 defectos recolectados de 36 operadores de uso extendido y encontraron que la mayoría se origina en una observación o un análisis incorrecto de los cambios de estado, y otra parte importante en la propia reconciliación. De ese hallazgo se sigue una decisión de ingeniería explícita para el proyecto, que consiste en componer operadores ya maduros para todo aquello que alguno de ellos cubra, y reservar el desarrollo propio para la lógica que ninguno provee.

Las extensiones que el proyecto compone cubren tres necesidades. El NVIDIA GPU Operator aprovisiona de forma declarativa el controlador de dispositivo, el *runtime* de contenedores y el complemento que expone la tarjeta gráfica al programador del clúster como recurso asignable, trabajo que de otro modo exigiría una instalación manual por nodo. Ese operador, y el exportador de métricas que lo acompaña en la Sección 4.7, leen estructuras propias de las tarjetas de ese fabricante, de modo que la plataforma queda acotada por diseño a nodos con aceleradores de NVIDIA, que son los que el laboratorio tiene instalados. KubeRay, sobre el *framework* de cómputo distribuido que Moritz et al. (2018) describen a partir de tareas asíncronas y actores con estado, aporta los recursos con los que un clúster de cómputo, un trabajo por lotes y un servicio de inferencia se declaran como objetos nativos, de modo que desplegar un modelo deja de ser la ejecución de un *script* sobre una máquina concreta. Una capa de admisión y encolamiento, por último, retiene los trabajos hasta que el clúster puede garantizarles la capacidad completa que solicitan, lo que evita que varias cargas parcialmente aprovisionadas se bloqueen entre sí. Ye et al. (2024) describen ese requisito en el entrenamiento distribuido, cuyos trabajos necesitan todas sus GPU asignadas a la vez, en la modalidad de todo o nada conocida como *gang scheduling*. Esa capa es, además, el punto donde se materializan las políticas de gobernanza de la Sección 4.5.

## 4.5. Gobernanza de recursos compartidos

Una infraestructura de cómputo acelerado compartida entre usuarios con necesidades desiguales plantea un problema de asignación que la literatura de sistemas ha abordado desde dos familias. La primera reparte la capacidad entre solicitantes considerados equivalentes y optimiza alguna métrica del clúster. Xiao et al. (2018) aprovechan la regularidad de las iteraciones del entrenamiento para repartir las GPU por turnos entre varios trabajos, migrarlos y agruparlos, y Gu et al. (2019) reducen el tiempo de finalización de los trabajos sin conocer de antemano su duración. Mahajan et al. (2020), Narayanan et al. (2020) y Qiao et al. (2021) incorporan la equidad entre trabajos o usuarios, con una noción de equidad en el tiempo de finalización, con políticas adaptadas a aceleradores heterogéneos y con un ajuste conjunto de los recursos y de los parámetros de cada entrenamiento, respectivamente. Ye et al. (2024) agrupan los objetivos de estos planificadores en eficiencia, equidad y cumplimiento de plazos.

La segunda familia establece jerarquías explícitas entre clases de solicitante y garantiza acceso preferente a las superiores mediante prioridad y, cuando hace falta, expropiación de recursos ya asignados. Borg, el gestor de clústeres de Google, sigue ese esquema. Cada trabajo tiene una

prioridad, una tarea de prioridad alta puede obtener recursos a costa de una de prioridad menor aunque eso implique expropiarla, y la cuota decide qué trabajos se admiten para su planificación (Verma et al., 2015). En un entorno académico, George (2020) describe un clúster universitario con pocas GPU que atiende a decenas de estudiantes y profesores, en el que se limita cuántos trabajos puede enviar y ejecutar cada estudiante y se da a los profesores una prioridad algo mayor que a los estudiantes. La elección entre ambas familias responde a una definición organizacional antes que técnica, y el contexto del IAsLab corresponde a la segunda. Los usuarios reservan capacidad según los roles que entrega SAAMFI, y cada rol tiene un nivel de prioridad que resuelve la contención de recursos por una regla declarada, de modo que una solicitud de un rol prioritario puede reclamar los recursos de una carga de menor prioridad cuando no hay capacidad libre. Una política de reparto proporcional entre iguales no satisface ese requisito, por bien que optimice la utilización agregada.

Dos mecanismos instrumentan la gobernanza que el proyecto necesita. La reserva anticipada aparta capacidad por adelantado para una carga específica durante una ventana de tiempo definida; Weitzel et al. (2025) documentan su introducción para las GPU de mayor demanda de un clúster Kubernetes multiusuario de escala nacional, y reportan una mejora marcada de la utilización promedio de las GPU, que sus operadores presentan como evidencia de la efectividad del sistema de reservas. El sobreaprovisionamiento controlado la complementa al asignar más capacidad nominal de la instalada, bajo el supuesto estadístico de que los solicitantes no agotan su cuota máxima de forma simultánea; el problema se ha formalizado como un empaquetado bajo restricciones probabilísticas (Cohen et al., 2019) y desplegado a escala de producción en múltiples centros de datos (Bashir et al., 2021), y su viabilidad descansa sobre la predictibilidad del consumo. Cortez et al. (2017) muestran, en las máquinas virtuales de Microsoft Azure, que ciertos comportamientos de una carga se mantienen consistentes a lo largo de su vida, de modo que su historia predice su comportamiento futuro, y usan esas predicciones para sobresuscribir servidores sin degradar el desempeño. El margen que el proyecto adopta, entre el 10 % y el 20 %, es una decisión de diseño del proyecto que estas fuentes no fijan.

En el plano de la implementación, las cuotas nativas que Kubernetes asocia a los espacios de nombres acotan el consumo agregado de procesador, memoria y dispositivos de un conjunto de cargas, y constituyen el punto de aplicación sobre el que se construyen la reserva y el margen de sobreasignación. La cuota define cuánto puede consumir un espacio de nombres, y la capa de admisión de la Sección 4.4 decide en qué momento un trabajo concreto obtiene ese consumo. Conviene señalar, por último, que gobernar cómputo acelerado en una institución educativa tiene rasgos propios, como el uso simultáneo de los equipos durante las clases o en las horas de mayor demanda que describe George (2020), y ese encuadre es el que justifica organizar la gobernanza del IAsLab alrededor de reservas y prioridades de rol.

## 4.6. Identidad federada y control de acceso

Aplicar una política de prioridad por rol exige saber con certeza quién solicita un recurso y qué rol ostenta. En entornos de cómputo en la nube, el control de acceso basado en roles resulta más escalable y adecuado que los modelos discrecional y obligatorio, y admite extensiones que incorporan a la decisión los atributos del solicitante y su grado de confianza (Li et al., 2015). La autenticación

federada resuelve el problema complementario al delegar la verificación de la identidad en un proveedor externo, que tras autenticar al usuario emite un *token* firmado con sus atributos, entre ellos sus roles; Fett et al. (2016) establecen mediante análisis formal las garantías de autorización, autenticación e integridad de sesión que ofrece OAuth 2.0 (*Open Authorization*) bajo ese esquema, una vez corregidas las vulnerabilidades que su propio análisis descubrió. La Universidad Icesi opera un proveedor institucional, SAAMFI, comparable en su función con productos como Keycloak o Auth0. Una plataforma apoyada en él hereda su taxonomía de roles y define por encima solo los que su propia operación requiere, como el de administrador, lo que evita comprometerse con una enumeración de perfiles académicos que pertenece al proveedor y puede cambiar sin intervención del proyecto.

La Figura 4.3 resume ese recorrido, desde la autenticación del usuario hasta los permisos con los que opera sobre el clúster.

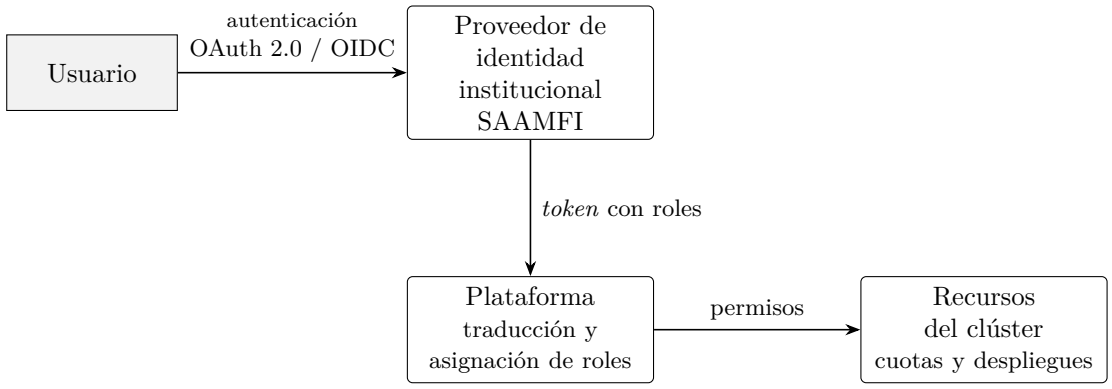



**Figura 4.3:** *Identidad federada y control de acceso. El usuario se autentica contra el proveedor institucional mediante OAuth 2.0 y OIDC, y recibe un* token *firmado con sus roles; la plataforma traduce esos atributos a sus propios roles y deriva de ellos los permisos con los que se accede a los recursos del clúster. Elaboración propia.*

Cuando las cuotas se aplican sobre el tráfico de inferencia, la literatura de microservicios ofrece el patrón conocido como *API gateway*, un punto único de entrada que da acceso a varias interfaces y al que resulta natural añadir funciones de descubrimiento de servicios, balanceo de carga, monitoreo y seguridad (Montesi and Weber, 2016). Sus variantes especializadas en modelos de lenguaje añaden límites basados en el consumo de *tokens*, que es la unidad en la que se contabiliza el uso de ese tipo de modelos. La revisión de esta literatura no localizó ninguna variante que aplique cuotas de forma unificada sobre cargas de lenguaje y cargas que no lo son, vacío relevante para el proyecto porque la plataforma debe gobernar cualquier carga de inteligencia artificial, y un mecanismo que solo sepa contar *tokens* deja fuera de la política a todo modelo cuya salida no se mida en esa unidad.

## 4.7. Observabilidad y telemetría de infraestructura acelerada

En los sistemas de microservicios, que suelen desplegarse sobre entornos de nube complejos, asegurar la observabilidad es esencial para entender su comportamiento y diagnosticar sus proble-

mas (Li et al., 2021). La observabilidad se apoya en tres señales complementarias, cada una con un propósito propio, que son los registros, las trazas y las métricas (Sridharan, 2018, como se citó en Li et al., 2021). En esos sistemas, los registros incluyen los de la aplicación y los del sistema, las trazas siguen el recorrido de cada solicitud entre los servicios que la atienden y las métricas abarcan las del sistema, como el consumo de procesador y de memoria, y las de la aplicación, como la latencia y la tasa de errores (Li et al., 2021). Las tres resultan necesarias porque responden preguntas distintas. Una métrica indica que algo se degradó, una traza indica dónde y un registro indica qué ocurrió en ese punto.

En un clúster de cómputo acelerado compartido, esa capacidad depende de telemetría *fine-grained* de *hardware*, porque la asignación de un recurso no revela cuánto se aprovecha. Jeon et al. (2019) advierten que, aunque la mayoría de las GPU de un clúster figuren asignadas, esa métrica por sí sola es engañosa, porque las GPU en uso alcanzaban en promedio solo alrededor de la mitad de su utilización. Weng et al. (2022) identifican la baja utilización de las GPU y las largas demoras en cola entre los retos de un clúster de producción con miles de GPU, y Gao et al. (2024) encontraron que la mayoría de los problemas de baja utilización que estudiaron podía corregirse con pocas modificaciones al código, en una plataforma que obtiene la utilización de cada GPU del gestor de GPU para centros de datos de NVIDIA (DCGM, *Data Center GPU Manager*) a través de Prometheus. La telemetría sirve también para anticipar fallas. Liu et al. (2023) estudian cómo predecirlas con modelos entrenados sobre datos recolectados de GPU en producción, entre ellos la temperatura, el consumo de potencia y la utilización de la GPU y de su memoria. A partir de esas fuentes y de los requisitos del laboratorio, el proyecto vigila la temperatura del núcleo y de la memoria, la utilización de la memoria de video (VRAM, *Video Random Access Memory*), el consumo de potencia y el *throughput* del bus PCIe (*Peripheral Component Interconnect Express*).

El ecosistema de Kubernetes instrumenta esas señales con un conjunto de componentes establecido. Prometheus recolecta métricas por consulta periódica a los puntos de exposición de cada componente, las almacena como series temporales identificadas por etiquetas y evalúa sobre ellas reglas de alerta. Thanos se apoya en Prometheus para conservar esas series a largo plazo y consultarlas desde un punto único, sin que la retención quede limitada al disco de una sola instancia. Loki agrega los registros que emiten los contenedores y los indexa por las mismas etiquetas, lo que permite pasar de una anomalía observada en una gráfica a los registros que la produjeron sin cambiar de sistema de coordenadas. Grafana consolida esas fuentes en paneles de visualización. El exportador DCGM de NVIDIA completa el arreglo al exponer a Prometheus las métricas de bajo nivel de las tarjetas aceleradoras, que residen en el *firmware* del dispositivo y que los demás componentes del conjunto no exponen. La Figura 4.4 resume ese arreglo.

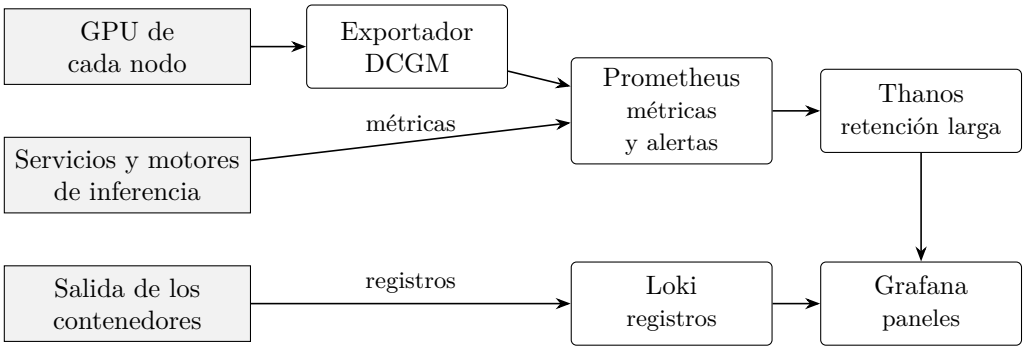



**Figura 4.4:** *Flujo de la telemetría en el clúster. Prometheus recolecta las métricas que exponen los servicios y, a través del exportador DCGM, las de las GPU; Thanos conserva esas series a largo plazo; Loki agrega los registros de los contenedores, y Grafana presenta esas fuentes en paneles. Elaboración propia.*

## 4.8. Marco conceptual y categorías de análisis

Los conceptos desarrollados en las secciones anteriores se emplean en el resto del documento con el significado operativo acotado que fija la Tabla 4.1, junto con el objetivo específico que cada uno sostiene. La tabla cumple el requisito del formato de relacionar cada concepto definido con los objetivos específicos, y lo concentra en un solo lugar para que la fundamentación de las secciones previas pueda leerse como un argumento continuo.

**Tabla 4.1:** *Definición operativa de los conceptos del marco teórico y objetivo específico que los emplea.*

| **Concepto** | **Definición operativa en este proyecto** | **Objetivo** |
|---|---|---|
| Carga de despliegue | Trabajo que ingresa al clúster a través de esta plataforma y pone en servicio un modelo ya entrenado para atender solicitudes de inferencia | Objetivo específico 3 |
| Motor de inferencia | Componente que carga los pesos de un modelo, gestiona la concurrencia y expone una interfaz de red, empaquetado como imagen de contenedor e intercambiable sin rediseñar la plataforma | Objetivo específico 3 |
| Recurso personalizado y operador | Tipo de objeto registrado en el servidor de API y controlador que lo reconcilia. Es el mecanismo por el que la plataforma declara clústeres de cómputo, trabajos y servicios de inferencia como objetos nativos del orquestador | Objetivo específico 3 |
| Admisión y encolamiento | Retención de un trabajo hasta que el clúster puede garantizarle la capacidad completa que solicita. Es el punto donde se materializa la política de prioridad | Objetivo específico 2 |
| Reserva con prioridad de rol | Asignación anticipada de capacidad por ventana de tiempo, con un margen de sobreasignación del 10 % al 20 % y con un nivel de prioridad por rol que permite reclamar los recursos de una carga de menor prioridad cuando no hay capacidad libre | Objetivo específico 2 |

| **Concepto** | **Definición operativa en este proyecto** | **Objetivo** |
|---|---|---|
| Identidad federada | Delegación de la autenticación en SAAMFI, cuyos roles la plataforma hereda y sobre los cuales define un único rol adicional de administración | Objetivo específico 2 |
| Telemetría de nodo | Captura de métricas de *hardware* y de carga de trabajo por nodo, visualizadas en paneles que muestran el estado de cada nodo del clúster | Objetivo específico 1 |
| Desempeño del servicio bajo carga concurrente | Nivel de concurrencia hasta el cual el servicio se sostiene, observado mediante el tiempo hasta el primer *token*, la latencia entre *tokens*, el *throughput*, el tiempo de espera en cola y la tasa de error | Objetivo específico 4 |

Las dos últimas filas fijan la categoría bajo la cual se analiza el resultado del proyecto, que es el desempeño del servicio bajo carga concurrente. La telemetría de nodo aporta las métricas de *hardware* sobre las que se interpreta una degradación observada durante la prueba, y la fila de desempeño reúne las métricas de inferencia y la tasa de error con las que se determina el nivel de concurrencia que el servicio sostiene. Ambas se recogen bajo pruebas de carga cuyo diseño, ejecución y análisis siguen la sistematización que Jiang and Hassan (2015) elaboran sobre la literatura de pruebas de carga en sistemas de gran escala. El Capítulo 6 desarrolla la aplicación concreta de esas pruebas dentro de la fase de evaluación empírica.

# 5. Estado del arte

## 5.1. Estrategia de revisión documental

El estado del arte presentado en este capítulo examina el conocimiento acumulado alrededor de la gestión, planificación, gobernanza y puesta en servicio de modelos de inteligencia artificial en clústeres de aceleradores de cómputo compartidos. Con el propósito de garantizar la pertinencia y solidez de las evidencias analizadas, la búsqueda documental se concentró en publicaciones arbitradas por pares en las principales bases de datos y bibliotecas digitales de ingeniería y ciencias de la computación, que incluyen IEEE Xplore, ACM Digital Library, las conferencias de USENIX y repositorios científicos de acceso abierto. La delimitación temática abarcó cuatro descriptores esenciales, que son la planificación y multiplexación de cargas de aprendizaje automático sobre unidades de procesamiento gráfico (GPU), las infraestructuras de aceleración en entornos universitarios, las plataformas de ciclo de vida de operaciones de aprendizaje automático (MLOps) y la optimización de motores de servicio de modelos de lenguaje e inferencia.

Los criterios de selección documental privilegiaron estudios empíricos, caracterizaciones de carga en producción y propuestas arquitectónicas publicadas a partir del año 2015. Como evidencia empírica se excluyeron los documentos sin revisión por pares y las propuestas de planificación anteriores al auge del aprendizaje profundo, que no contemplan las particularidades del acceso a la memoria de video. La documentación oficial de las herramientas se empleó solo para describir sus capacidades. Las plataformas comerciales en nube pública se mantuvieron en el análisis, aunque no permiten una operación local, para dejar explícita la razón por la que se descartan como solución. Como resultado de este proceso de depuración, la literatura se agrupó en cuatro vertientes de alternativas de solución previas, que abarcan seis familias de soluciones, y en la comparación de motores de inferencia, que son componentes de la plataforma y por eso quedan fuera del conjunto de alternativas. Las características, fortalezas y limitaciones técnicas de cada una se analizan a continuación frente a los requisitos del Laboratorio de Inteligencia Artificial y Arquitectura de Software (IAsLab) de la Universidad Icesi. Cuando una familia se describe por su documentación oficial y no por un estudio arbitrado, la cita remite a esa documentación.

## 5.2. Gestión y planificación en clústeres de aceleradores a gran escala

La administración de aceleradores en clústeres multiusuario de gran tamaño ha sido abordada de manera extensa por la industria tecnológica y la investigación en sistemas distribuidos. Jeon et al. (2019) analizaron una traza de dos meses de Philly, la plataforma de Microsoft para el entrenamiento de modelos de aprendizaje profundo, que procesó cerca de 100 000 trabajos de cientos de usuarios. Los autores evidenciaron que, a pesar de que la mayoría de las GPU figuraban asignadas a los usuarios, la utilización real del *hardware* en uso alcanzaba únicamente cerca del 52 % en promedio,

al tiempo que cerca del 30 % de las tareas se cancelaba o terminaba de forma anómala debido a fallos. Ese desajuste muestra que una GPU asignada no equivale a una GPU aprovechada.

En un sentido similar, Gu et al. (2019) evaluaron el comportamiento de un clúster de producción y observaron que los planificadores tradicionales concebidos para el procesamiento de grandes volúmenes de datos provocan prolongadas demoras en cola y un bajo desempeño global cuando ejecutan tareas de aprendizaje profundo. Para contrarrestar esta limitación, desarrollaron Tiresias, un gestor de clúster que reduce el tiempo de finalización de los trabajos con dos algoritmos bidimensionales, uno basado en el índice de Gittins cuando se dispone de información parcial y otro que no requiere información alguna sobre su duración. Por su parte, Weng et al. (2022) analizaron una traza de dos meses en un clúster de producción con más de 6 000 GPU heterogéneas en Alibaba, y explicaron los desafíos que esa carga plantea a la planificación, entre ellos la baja utilización de las GPU, los largos retardos en cola, las tareas difíciles de ubicar por exigir GPU de gama alta, el desbalance de carga entre máquinas heterogéneas y un posible cuello de botella en los procesadores. Para abordar la gobernanza y equidad en el reparto de recursos heterogéneos, Mahajan et al. (2020) propusieron Themis, un esquema de subastas que busca asegurar a largo plazo la equidad en el tiempo de finalización de las cargas de aprendizaje automático, mientras que Narayanan et al. (2020) formularon Gavel, un planificador consciente de la heterogeneidad de los aceleradores que generaliza distintas políticas de planificación al expresarlas como problemas de optimización. Asimismo, Gao et al. (2024) encontraron que los problemas de baja utilización de las GPU se originan en la lógica del código de los trabajos, por cómputo insuficiente en la GPU o por interrupciones de tareas ajenas a ella, en una plataforma que obtiene la utilización de cada GPU del gestor DCGM de NVIDIA a través de Prometheus.

A pesar de sus aportes matemáticos y arquitectónicos, estas propuestas industriales no resuelven el problema que enfrenta el IAsLab. Dichos sistemas fueron diseñados para centros de datos empresariales de hiperescala, provistos de cientos o miles de aceleradores enlazados por redes de conmutación de alta velocidad. El IAsLab, en cambio, dispone de 31 estaciones de trabajo con GPU en la sala 104M, un aula de docencia que comparte esos equipos con la investigación, conectadas por una red de área local estándar. En segundo término, estos gestores se enfocan primordialmente en la fase de entrenamiento distribuido por lotes (*batch*), orientando sus heurísticas a minimizar el tiempo de finalización de trabajos largos. Por el contrario, la necesidad del IAsLab radica en la etapa posterior del ciclo, consistente en el despliegue interactivo y continuo de modelos para su consumo mediante llamadas a servicios de red con latencias acotadas. Finalmente, operar estas arquitecturas requiere personal especializado, un recurso escaso en los laboratorios universitarios, que suelen administrar sus clústeres compartidos con personal limitado (Xu et al., 2025).

## 5.3. Infraestructuras de aceleración en entornos universitarios

En el ámbito académico, las universidades han recurrido históricamente a herramientas consolidadas de computación de alto rendimiento (HPC, *High Performance Computing*) para compartir recursos escasos. Un esquema simple y común consiste en otorgar a los usuarios acceso directo por consola a las máquinas con GPU a través de SSH (*Secure Shell*), y otro, en administrar las máquinas con un planificador de colas como Slurm, un sistema de gestión de clústeres y planificación de tra-

bajos que asigna recursos y arbitra su disputa mediante una cola de trabajos pendientes (SchedMD, 2026). Xu et al. (2025) advierten que muchas universidades carecen de la experiencia necesaria para aprovechar estos clústeres compartidos y adoptan una herramienta conocida con configuraciones subóptimas, con lo que terminan subutilizando recursos ya limitados. Según los autores, el acceso directo por consola carece de mecanismos de aislamiento y gestión de recursos, suscita inquietudes de estabilidad y seguridad, y obliga a los usuarios a configurar a mano sus entornos y a coordinar el uso de las GPU por otros canales, lo que puede producir conflictos entre sesiones concurrentes y dificulta un reparto equitativo.

Para superar estas fallas en campus educativos, la literatura reciente documenta plataformas adaptadas a la comunidad universitaria. Xu et al. (2025) presentaron SING, un clúster de aprendizaje automático en campus que gestiona más de 160 GPU para más de 480 usuarios activos, y señalan que compartir estos recursos de forma efectiva es esencial para maximizar tanto su utilización como su accesibilidad. En una escala geográfica mayor, la National Research Platform reúne infraestructura multiusuario sobre Kubernetes que atiende a investigadores de 50 campus universitarios (Weitzel et al., 2025). En dicha plataforma, la incorporación de un sistema de reservas sobre las tarjetas de alta gama elevó la utilización promedio de las GPU del 21.49 % al 31.37 %, mejora que sus operadores presentan como evidencia de la efectividad del sistema de reservas.

Aunque estas soluciones se aproximan al contexto académico, no resultan directamente aplicables a los requerimientos del IAsLab, y cada una resuelve una parte del problema. La National Research Platform autentica a sus usuarios con identidad federada y los restringe a los espacios de nombres de sus grupos de investigación (Weitzel et al., 2025), SING aplica cuotas de recursos por usuario (Xu et al., 2025), y el clúster que describe George (2020) limita los trabajos de cada estudiante y da a los profesores una prioridad algo mayor. Slurm, por su parte, ofrece encolamiento y planificación de trabajos, y Xu et al. (2025) señalan que carece de soporte para aprovisionar los entornos de dependencias que exigen los trabajos de aprendizaje automático, que su nodo de acceso puede convertirse en un punto único de falla y que su asignación de recursos de grano grueso puede fragmentarlos. Ese esquema de trabajos encolados no corresponde al despliegue persistente de servicios de inferencia en contenedores que requiere el IAsLab. La National Research Platform, por su parte, opera como una federación científica de gran escala que no contempla una sala de docencia cuyas estaciones de trabajo deben seguir disponibles para las clases presenciales. Ninguna de estas soluciones reúne en una sola plataforma el despliegue de servicios de inferencia, la reserva de cupos con prioridad según los roles que entrega el proveedor de identidad de cada institución y la convivencia con las clases en las mismas estaciones, que es la combinación que requiere el IAsLab.

## 5.4. Ecosistemas de MLOps y plataformas de ciclo de vida

Tras la identificación de los costos ocultos de mantener sistemas de aprendizaje automático en producción (Sculley et al., 2015), la literatura propuso el paradigma de MLOps para el ciclo de vida de estos sistemas (Kreuzberger et al., 2023; Eken et al., 2026), aunque Eken et al. (2026) advierten que su conceptualización unificada sigue sin lograrse. Kreuzberger et al. (2023) recogen, entre los servicios en la nube para poner modelos en servicio, Microsoft Azure ML, AWS SageMaker, IBM Watson Studio y Google Vertex AI, y Eken et al. (2026) señalan, con base en uno de los

estudios que revisan, que las plataformas gestionadas en la nube, como Amazon SageMaker, dan soporte a la preparación de datos, el entrenamiento y el seguimiento de experimentos, el despliegue automatizado sobre infraestructura gestionada y el monitoreo durante todo el ciclo de vida.

A pesar de su funcionalidad integral, las plataformas en nube comercial quedan descartadas para el IAsLab por una razón estructural y por barreras económicas directas. En el aspecto estructural, el proyecto exige operar de forma estricta en modalidad local (*on-premise*) sobre el *hardware* físico ya instalado en la sala 104M del laboratorio. En el aspecto económico, los esquemas de facturación por consumo en nube imponen costos recurrentes que la comunidad estudiantil y los grupos de investigación podrían no poder financiar de manera continua, lo que marginaría a los usuarios con menores recursos. Eken et al. (2026) recogen, además, que usar plataformas completamente gestionadas implica perder control *fine-grained* sobre la pila tecnológica, y que los servicios de la plataforma pueden no cubrir los requisitos especializados de muchas empresas.

En el ámbito del *software* libre desplegable sobre Kubernetes, Kreuzberger et al. (2023) mencionan KServing de Kubeflow, TensorFlow Serving y Seldon como marcos de servicio de modelos compatibles con Kubernetes, y describen Kubeflow como una plataforma de aprendizaje automático de extremo a extremo basada en Kubernetes, en la que cada componente se empaqueta en un contenedor. Estas herramientas cubren el servicio de modelos, pero su adopción tiene un costo. Eken et al. (2026) recogen que, pese a la potencia de herramientas como Kubeflow, el alto costo de infraestructura, la experiencia que exige operar sus canalizaciones y la sobrecarga de instalación y mantenimiento frenan su adopción, en especial en las etapas iniciales de los proyectos y en los proyectos impulsados por empresas pequeñas. En cuanto al reparto de recursos, Kubeflow organiza a sus usuarios en perfiles que envuelven un espacio de nombres de Kubernetes, con un propietario y colaboradores de solo lectura o de edición, y permite crear opcionalmente una cuota de recursos para cada perfil (Kubeflow, 2026). Su documentación no describe niveles de prioridad por rol ni reservas de cupos. Además, Xu et al. (2025) señalan que Kubernetes carece de soporte nativo para el control de acceso por usuario y el encolamiento de trabajos, que exigen herramientas adicionales. Por esa razón, el IAsLab necesita construir esa gobernanza sobre el orquestador.

## 5.5. Pasarelas de inferencia y gobernanza de interfaces de programación

Con la proliferación de los LLM, surgieron herramientas intermediarias enfocadas en estandarizar el consumo de servicios de inferencia mediante pasarelas de interfaz de programación de aplicaciones (*API Gateways*), entre las que destaca LiteLLM. Este enfoque se sustenta en patrones reconocidos de microservicios (Montesi and Weber, 2016), en el que la pasarela actúa como un componente intermediario (*proxy*) entre los clientes y los servicios que consumen.

La documentación de LiteLLM lo describe como una biblioteca de código abierto que ofrece una interfaz unificada para invocar más de cien modelos de lenguaje con el formato de OpenAI, con lógica de reintentos y de respaldo entre varios despliegues, y como una pasarela autoalojada con llaves virtuales, seguimiento de costos y presupuestos por llave, equipo o usuario (LiteLLM, 2026b). Su proxy permite además fijar límites de tokens por minuto, de peticiones por minuto y de peticiones en paralelo por llave o equipo (LiteLLM, 2026a). Esa gobernanza se expresa en gasto,

tokens y peticiones, de modo que no aborda los cupos de GPU ni la prioridad de una carga sobre otra.

LiteLLM opera en la capa de aplicación sobre el protocolo de transferencia de hipertexto (HTTP, *Hypertext Transfer Protocol*), y su documentación describe el acceso a modelos que ya se encuentran servidos, sin describir el despliegue ni la gestión de las instancias que los sirven. Por ello, una pasarela como esta no puede constituir el núcleo de la plataforma, porque no crea, reinicia ni destruye servicios de inferencia sobre el orquestador de contenedores, y su telemetría se limita a costos y registros de peticiones, sin la del estado de las tarjetas aceleradoras. Estas son lecturas de los autores a partir de lo que la documentación describe y de lo que no describe.

## 5.6. Motores de inferencia como componentes de la plataforma

Los motores de inferencia son los componentes que la plataforma despliega y orquesta, por lo que no se tratan como una alternativa de solución al problema del IAsLab. Esta sección compara sus compromisos de diseño para sustentar una decisión de arquitectura. El despliegue eficiente de modelos de lenguaje sobre aceleradores con memoria limitada exige técnicas avanzadas de administración de memoria y planificación de peticiones. En la literatura especializada se identifican dos vertientes con compromisos de diseño marcadamente diferenciados, representadas por vLLM y *llama.cpp*. Por una parte, vLLM introduce el algoritmo PagedAttention, que gestiona el almacén de claves y valores (KV *cache*) mediante principios análogos a la paginación de memoria virtual en sistemas operativos, con lo cual alivia la fragmentación interna, elimina la externa y logra un desperdicio casi nulo de la memoria de la caché (Kwon et al., 2023). Al combinarse con planificación a nivel de iteración (*iteration-level scheduling*) y procesamiento de lotes continuo (Yu et al., 2022; Miao et al., 2025), este motor maximiza la tasa de procesamiento global (*throughput*) cuando atiende múltiples peticiones concurrentes.

No obstante, en las pruebas que el laboratorio realizó sobre las estaciones de la sala 104M, equipadas cada una con una GPU NVIDIA RTX 4090 de 24 GB, vLLM no produjo buenos resultados, y la mejor opción fue una compilación de *llama.cpp* ajustada a ese *hardware*. *llama.cpp* se plantea como objetivo la inferencia de modelos con una configuración mínima, y su implementación en C/C++ no tiene dependencias y admite cuantización a enteros de entre 1,5 y 8 bits, con núcleos CUDA propios para las GPU de NVIDIA (ggml-org, 2026). Con pesos cuantizados en formato GGUF, esa combinación permite alojar modelos de 7 000 a 14 000 millones de parámetros dentro de esa memoria. La cuantización posentrenamiento reduce la precisión numérica de los pesos procurando mantener una pérdida mínima de rendimiento (Frantar et al., 2023; Chen et al., 2026). Su documentación, sin embargo, describe un motor de inferencia y no políticas multiusuario de reparto ni herramientas de gobernanza de clúster, de modo que esas capacidades quedan fuera de lo que ofrece por sí solo.

Este análisis comparativo sustenta una decisión central de ingeniería para el proyecto. En lugar de acoplar la arquitectura a un motor específico, la plataforma debe tratar a los motores de inferencia como componentes empaquetados en imágenes de contenedor e intercambiables según el perfil de la carga solicitada. Esta modularidad permite aprovechar la baja latencia y el consumo

reducido de *llama.cpp* para sesiones interactivas individuales o modelos cuantizados en GGUF, al tiempo que conserva la capacidad de incorporar vLLM u otros motores especializados cuando la concurrencia o la naturaleza del modelo así lo demanden.

## 5.7. Síntesis comparativa y vacíos identificados

La Tabla 5.1 resume la caracterización de las alternativas de solución analizadas en las secciones previas frente a los requerimientos operacionales y contextuales de la infraestructura del IAsLab en la Universidad Icesi. La evaluación contempla cinco criterios técnicos y organizacionales determinantes para la viabilidad de la plataforma en este escenario, que comprenden la capacidad de operar de forma estricta en infraestructura local (*on-premise*) sin dependencia de nubes comerciales, el soporte agnóstico para cargas diversas de inteligencia artificial (tanto modelos de lenguaje como arquitecturas tradicionales de aprendizaje automático), la existencia de un sistema de gobernanza de cuotas acoplado a la identidad académica institucional (SAAMFI), la observabilidad de aceleradores *fine-grained* en tiempo real basada en métricas de *hardware*, y el desacoplamiento arquitectónico frente al motor de inferencia subyacente.

**Tabla 5.1:** *Comparación de alternativas previas frente a los requerimientos de la plataforma IAsLab ORCHID. Valoración de los autores a partir de las Secciones 5.2 a 5.5 y de las fuentes que en ellas se citan.*

| **Enfoque o solución** | **Operación *on-premise*** | **Cargas LLM y ML** | **Gobernanza por rol** | **Telemetría GPU en tiempo real** | **Desacoplamiento de motor** |
|---|---|---|---|---|---|
| Clústeres HPC por colas (Slurm) | Total | Parcial (trabajos por lotes, sin soporte de dependencias de ML) | Parcial (límites y prioridad por usuario) | Sin integración documentada | No aplica (ejecuta trabajos, no servicios) |
| Gestores industriales de hiperescala | Inviable (requiere gran escala) | Parcial (foco en entrenamiento) | Parcial (equidad entre usuarios, no por rol institucional) | Disponible (Prometheus y DCGM en una plataforma analizada) | No aplica (planifican entrenamiento) |
| Clústeres de campus (SING, NRP, clúster de clase) | Total | Parcial (trabajos de ML por lotes) | Parcial (cuotas, reservas o prioridad por tipo de usuario, según el clúster) | Parcial (monitoreo de utilización de GPU) | No aplica (trabajos por lotes) |
| Plataformas MLOps en nube comercial | Nula (exclusivo nube) | Amplia (datos, entrenamiento, despliegue y monitoreo) | Limitada (se pierde control fino de la pila) | Monitoreo del ciclo de vida gestionado | Parcial (modular y personalizable según el proveedor) |
| Ecosistemas MLOps en Kubernetes (Kubeflow, Seldon) | Total | Parcial (marcos de servicio y canalizaciones, con costo y experiencia de operación) | Parcial (perfiles con cuota opcional, sin prioridad por rol documentada) | Sin integración documentada | Parcial (varios marcos de servicio) |

| **Enfoque o solución** | **Operación *on-premise*** | **Cargas LLM y ML** | **Gobernanza por rol** | **Telemetría GPU en tiempo real** | **Desacoplamiento de motor** |
|---|---|---|---|---|---|
| Pasarelas de inferencia (LiteLLM) | Total (autoalojada) | Parcial (acceso unificado a modelos ya servidos) | Parcial (presupuestos y límites de tasa por llave, equipo o usuario) | Limitada (costos y registros de peticiones) | Parcial (unifica el acceso, no despliega) |
| **IAsLab ORCHID (Propuesta)** | **Total (sala 104M)** | **Total (LLM y ML)** | **Total (reserva por rol SAAMFI)** | **Total (DCGM, Prometheus, Loki)** | **Total (contenedores intercambiables)** |

A partir de la comparación efectuada, se identifican tres vacíos en el conocimiento aplicado y en las herramientas de ingeniería existentes. El primero es que las soluciones revisadas se reparten entre los clústeres industriales de gran escala, los clústeres HPC de investigación por lotes y los clústeres dedicados de campus, como SING o la National Research Platform, y ninguna aborda el caso de un laboratorio cuyas estaciones de trabajo con aceleradores de gama media-alta se reparten entre la docencia presencial y la investigación aplicada. El segundo es la ausencia de una plataforma que integre la gobernanza del acceso mediante reserva de cupos con prioridad por rol con el aprovisionamiento de servicios de inferencia desacoplados. Las soluciones revisadas resuelven por separado la identidad, las cuotas o la prioridad, sin reunirlas alrededor del servicio de inferencia. El tercero es acercar la telemetría de *hardware* de bajo nivel, que herramientas como Prometheus y DCGM ya recolectan en la plataforma empresarial que analizan Gao et al. (2024), a una interfaz de autoservicio con la que los administradores de una sala académica puedan vigilar la saturación de la memoria de video y el estado térmico de los nodos compartidos.

La plataforma IAsLab ORCHID se proyecta precisamente para cerrar estos vacíos en la Universidad Icesi. Al orquestar contenedores sobre Kubernetes en la sala 104M, implementar un sistema de reserva de cupos vinculado a los roles de SAAMFI con prioridad por rol, instrumentar el clúster con Prometheus, Thanos, Loki, Grafana y el exportador DCGM de NVIDIA, y mantener desacoplado el motor de servicio de inferencia, la propuesta articula las fortalezas observadas en el estado del arte y busca resolver las limitaciones estructurales que impiden a las soluciones preexistentes operar en este contexto institucional.

# 6. Metodología

Este capítulo formula y justifica la metodología de ingeniería adoptada para resolver el problema planteado en el Capítulo 2, describe la estructura lógica del proceso de investigación, desde la recolección de datos hasta la presentación de resultados, expone el proceso de elicitación y especificación de requerimientos que sustentan los objetivos específicos del proyecto (cuyo catálogo detallado se compila en el Anexo D), remite al Capítulo 7 para las fases de desarrollo y el cronograma de ejecución, y al Anexo E para el análisis exhaustivo de riesgos y limitaciones que pueden afectar su alcance, cronograma y calidad.

## 6.1. Estrategia metodológica

El proyecto adopta un proceso de desarrollo iterativo e incremental de tipo Scrum, siguiendo la *Scrum Guide* de Schwaber and Sutherland (2020), organizado en *sprints* de tres semanas. Esta elección responde a que el alcance del proyecto, cuatro objetivos específicos con dependencias técnicas parciales entre sí, se beneficia de ciclos cortos de construcción y retroalimentación, en los que cada *sprint* entrega un incremento verificable y permite reordenar el trabajo restante conforme avanza el desarrollo y conforme lo permite la disponibilidad de los nodos de cómputo del IAsLab, en lugar de comprometer de antemano una secuencia rígida de actividades como exigiría un modelo de desarrollo en cascada.

La duración de tres semanas por *sprint* responde a la naturaleza del trabajo de las primeras iteraciones. Levantar el clúster comprende una cadena de actividades, entre ellas desplegar Kubernetes, primero sobre un prototipo en máquinas virtuales y después sobre las estaciones de trabajo de la sala 104M, configurar la red, los nodos de control y los nodos de trabajo, aprovisionar el acceso a las unidades de procesamiento gráfico y, sobre esa base ya estable, integrar las extensiones que la plataforma necesita. Cada una de ellas depende de que la anterior quede estable, de modo que una iteración más corta obligaría a cerrar *sprints* sin un clúster verificable. El equipo adoptó por eso una cadencia de tres semanas, que permite terminar cada iteración con infraestructura funcionando y comprobable.

Los artefactos y ceremonias que el equipo emplea de forma efectiva son un tablero en Jira para el *backlog* y el seguimiento de las iteraciones, un repositorio Bitbucket para el control de versiones y los archivos de configuración de la infraestructura, y una reunión semanal de seguimiento con el tutor del proyecto, en la que se revisa el avance de la iteración en curso y se ajusta el alcance de la siguiente. No se documentan otras ceremonias de Scrum, por ejemplo el *daily scrum* o las retrospectivas formales, porque el equipo no las practica de forma sistemática; el punto de control periódico del proyecto es la reunión semanal con el tutor.

La estructura lógica del proceso de investigación se organiza en cuatro etapas. La recolección

de datos se apoya en dos fuentes instrumentadas por el propio sistema, que son la telemetría de *hardware* e inferencia que produce el módulo de observabilidad y los resultados de las pruebas de carga que ejecutan *scripts* de Python externos a la plataforma sobre los nodos de inferencia. La organización y sistematización de estos datos consiste en consolidar las métricas de *hardware* y de inferencia en formatos comparables entre fases y entre motores de inferencia evaluados. El análisis contrasta estas métricas contra los objetivos específicos formulados, determinando en qué medida cada módulo construido resuelve la carencia que motivó su desarrollo. Finalmente, la interpretación y presentación de resultados traduce los hallazgos del análisis en conclusiones sobre el nivel de concurrencia que la plataforma sostiene.

La evaluación empírica prevista en el objetivo específico 4 ejecuta pruebas de carga con usuarios concurrentes mediante *scripts* de Python externos a la plataforma, para no complejizar su desarrollo. El Capítulo 7 (Sección 7.1) ubica esa evaluación en la fase correspondiente, y el Capítulo 4 presenta el respaldo metodológico de la prueba de carga.

## 6.2. Elicitación y especificación de requerimientos

Los requerimientos del proyecto se levantaron a partir de las sesiones de elicitación con el personal del IAsLab, que sirvieron como insumo técnico y operativo. Después de revisarlos y ajustarlos, se contrastaron contra los objetivos específicos para validar y acotar el alcance propuesto de la solución. Con ese alcance fijado, el equipo procedió a su refinamiento y priorización, los cuales fueron validados por el tutor del proyecto. Para facilitar que un evaluador verifique el cumplimiento sistemático de cada objetivo contra los entregables descritos en el Capítulo 8, los requerimientos se agruparon por objetivo específico y se codificaron bajo la convención Rk-nn, donde k indica el objetivo y nn el consecutivo correspondiente. El catálogo detallado con los 110 requerimientos funcionales y no funcionales se presenta en el Anexo D, agrupado en los mismos subsistemas funcionales con los que se especificó la plataforma (Secciones D.1 a D.4).

## 6.3. Riesgos y limitaciones del proyecto

El despliegue de la plataforma sobre infraestructura de cómputo compartida conlleva incertidumbres técnicas y de gestión derivadas de las restricciones operativas y supuestos del proyecto. Para salvaguardar el alcance, el cronograma y la calidad de la solución, el equipo caracterizó los riesgos técnicos y metodológicos, identificando la dimensión que comprometen y formulando estrategias preventivas de mitigación. El análisis detallado de cada uno de los riesgos identificados y sus medidas de mitigación se presenta en el Anexo E.

# 7. Fases de desarrollo y cronograma

Este capítulo define las fases de desarrollo del proyecto a partir de la cadena de dependencias técnicas que articula los objetivos específicos formulados en el Capítulo 3, establece la duración estimada de cada una de ellas y presenta el cronograma general de ejecución mediante un diagrama de Gantt, señalando la distribución temporal de los *sprints*, los hitos de entrega y la ruta crítica que rige la construcción de la plataforma.

## 7.1. Fases de desarrollo del proyecto

Las fases de desarrollo del proyecto guardan correspondencia directa con los objetivos específicos formulados en el Capítulo 3 y se estructuran según una cadena estricta de dependencias técnicas. El proceso inicia con la puesta en marcha del clúster, primero sobre un prototipo en máquinas virtuales y después sobre las estaciones de la sala 104M (Sección D.3, grupo «Clúster base») y el despliegue de los motores de inferencia. Sobre esa infraestructura se instrumenta el módulo de observabilidad (Sección D.1), necesario para recolectar telemetría y métricas operativas antes de aplicar restricciones de recursos. Posteriormente, se implementa la gobernanza de cuotas dentro del clúster (Sección D.2, grupos «Identidad y control de acceso», «Cuotas y aislamiento» y «Admisión, prioridad y reservas») y se construye la plataforma web de despliegue y administración, la cual expone la interfaz de despliegue del objetivo específico 3 y el módulo administrativo del objetivo específico 2 sobre las capacidades ya estabilizadas. El ciclo culmina con la evaluación empírica del sistema integrado mediante las pruebas de carga del grupo «Medición de desempeño» (Sección D.4). La Tabla 7.1 sintetiza esta secuencia con la duración estimada de cada fase en *sprints* de tres semanas.

**Tabla 7.1:** *Fases de desarrollo del proyecto, su correspondencia con los objetivos específicos y su duración.*

| **Fase** | **Actividades principales** | **Objetivo específico** | **Duración** |
|---|---|---|---|
| Puesta en marcha del clúster | Grupos «Clúster base», «Orquestación de cargas» y «Motores y modelos» de la Sección D.3 | Objetivo específico 3 | 2 *sprints* |
| Observabilidad de infraestructura | Los cuatro grupos de la Sección D.1 | Objetivo específico 1 | 2 *sprints* |
| Gobernanza de cuotas | Grupos «Identidad y control de acceso», «Cuotas y aislamiento» y «Admisión, prioridad y reservas» de la Sección D.2 | Objetivo específico 2 | 2 *sprints* |

| Fase | Actividades principales | Objetivo específico | Duración |
|---|---|---|---|
| Plataforma de despliegue y administración | Grupo «Interfaz de despliegue» de la Sección D.3 y grupo «Módulo administrativo» de la Sección D.2 | Objetivo específico 3, objetivo específico 2 | 3 *sprints* |
| Evaluación empírica | Grupo «Medición de desempeño» de la Sección D.4 | Objetivo específico 4 | 1 *sprint* |

## 7.2. Cronograma

El proyecto inicia el 10 de agosto de 2026 y finaliza el 31 de mayo de 2027. Entre el 10 de agosto y el 20 de septiembre de 2026 el equipo dedica una etapa previa a la formulación del anteproyecto, el levantamiento de requisitos con el laboratorio y la definición de la arquitectura propuesta. Los *sprints* de desarrollo comienzan el 21 de septiembre de 2026 y se suceden en ciclos de tres semanas hasta el cierre del proyecto. Las actividades de cada *sprint* se derivan de los grupos de requerimientos presentados en las Secciones D.1 a D.4, y las asume el equipo del proyecto en su conjunto, razón por la cual el cronograma omite una columna de responsable. La Figura 7.1 presenta el cronograma completo, con las fechas de cada *sprint*, los grupos de requerimientos que atiende y el hito de entrega del anteproyecto. Como cada fase depende de que la anterior quede estable, las cinco fases forman una sola cadena que constituye en conjunto la ruta crítica del proyecto, de modo que el retraso de cualquier *sprint* desplaza el cierre.

La ejecución del proyecto se estructura en dos grandes etapas consecutivas. La primera etapa abarca de agosto a diciembre de 2026, periodo en el que se consolida la formulación del anteproyecto y se ejecutan los cuatro primeros *sprints* de desarrollo, correspondientes a la puesta en marcha del clúster de cómputo y a la instrumentación de observabilidad de infraestructura. La segunda etapa comprende de enero a mayo de 2027, periodo durante el cual se implementan el sistema de gobernanza de cuotas, la plataforma de despliegue y administración, y la evaluación empírica del sistema, culminando con el informe final del proyecto.

El clúster se despliega en las estaciones de la sala 104M en diciembre de 2026. Hasta entonces, octubre y noviembre se trabajan sobre una prueba de concepto (POC, *Proof of Concept*), un prototipo montado en máquinas virtuales de un servidor del laboratorio, donde se construyen y verifican los componentes antes de llevarlos a la sala. El cambio de lugar no altera el orden de las fases ni su duración, porque la configuración versionada del clúster se traslada a la sala una vez que el prototipo queda verificado.

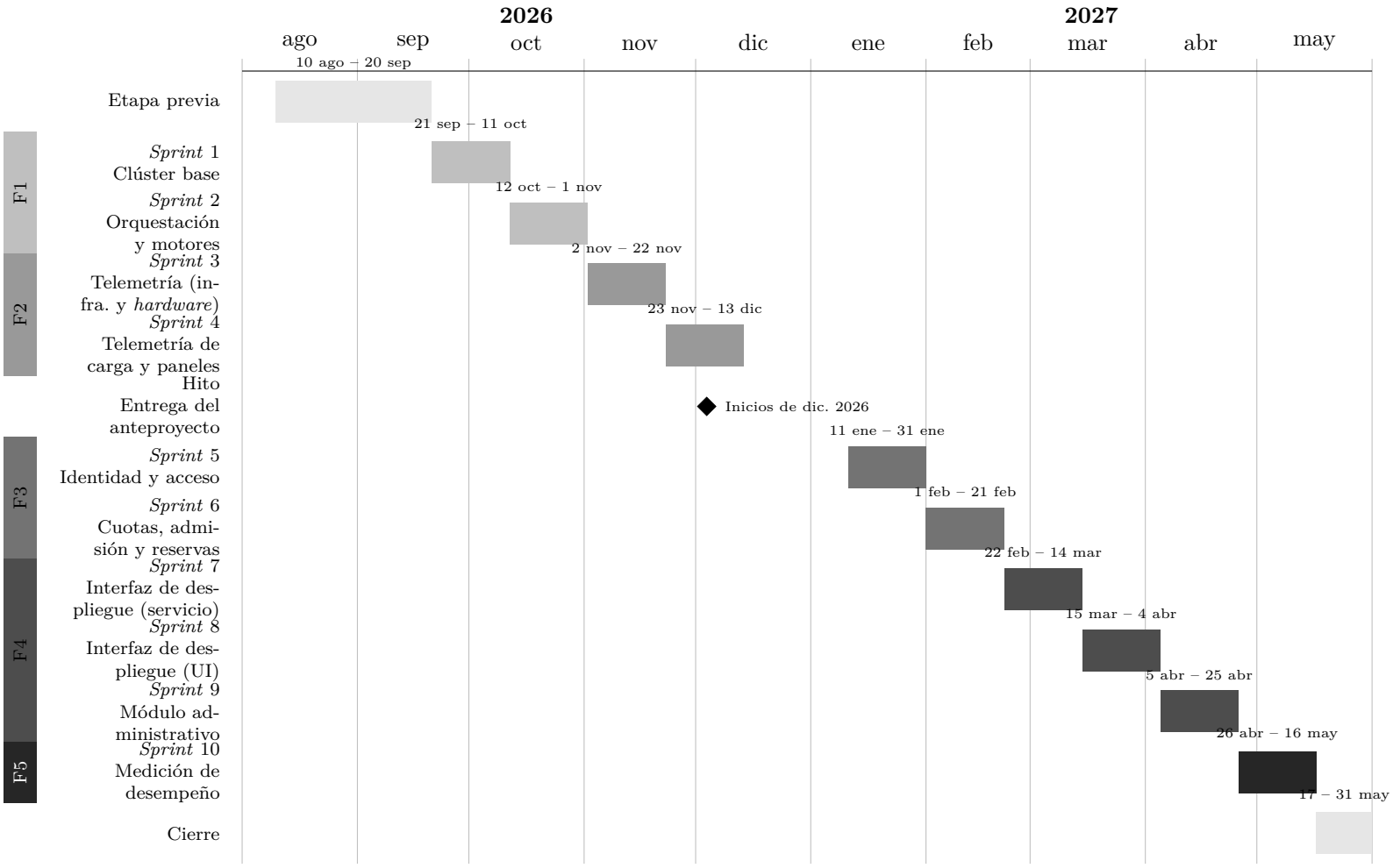



**Figura 7.1:** *Diagrama de Gantt del proyecto. Cada barra indica sus fechas y, para los* sprints*, el grupo de requerimientos que atiende, en el orden de las Secciones D.1 a D.4; el tono lateral identifica la fase (leyenda F1–F5) y el rombo marca la entrega del anteproyecto a inicios de diciembre de 2026. Las cinco fases forman una sola cadena de dependencias y constituyen en conjunto la ruta crítica del proyecto. Elaboración propia.*

# 8. Contribución y resultados del proyecto de grado

## 8.1. Aportes relacionados con el objeto del proyecto

La plataforma descrita en los capítulos anteriores incide directamente sobre el problema formulado en el Capítulo 2, que reúne la interrupción del ciclo de vida de los modelos tras el entrenamiento y la gestión manual de una infraestructura compartida que no se gobierna ni se observa. Su primer aporte es de naturaleza institucional y económica. Al habilitar el despliegue de modelos ya entrenados sobre el mismo *hardware* que hoy se usa para el entrenamiento, el proyecto maximiza el retorno de la inversión realizada en la infraestructura del laboratorio, en lugar de dejarla limitada a la fase experimental del ciclo de vida. El sistema de reserva de cupos con prioridad por rol, acoplado a los roles institucionales, amplía el acceso a las unidades de procesamiento gráfico entre estudiantes y profesores de la universidad, con reglas de prioridad declaradas para cada rol. Esta misma capacidad reduce la necesidad de adquirir créditos de cómputo en nubes públicas comerciales para sostener cargas de trabajo que la infraestructura *on-premise* del laboratorio puede absorber, argumento que se desarrolla en el Capítulo 1.

En el plano académico, el proyecto habilita la ejecución de proyectos avanzados en áreas prioritarias para el macroproyecto, como Industria 4.0 y e-Salud, al permitir que arquitecturas de gran escala, en particular los LLM, pasen de la fase de entrenamiento a un servicio de inferencia consumible dentro de la misma plataforma institucional. Esta capacidad tiene además una implicación de privacidad y gobernanza de datos, porque al mantener el procesamiento de inferencia sobre servidores propios del IAsLab, en lugar de delegarlo en proveedores externos, la información asociada a estos proyectos de investigación permanece bajo el control administrativo de la universidad. Finalmente, en el plano operativo, el módulo de telemetría y la administración centralizada de cuotas reducen la carga cognitiva y administrativa que hoy recae sobre los administradores del laboratorio, quienes actualmente carecen de visibilidad *fine-grained* sobre el consumo real de la infraestructura para diagnosticar saturaciones o conflictos de uso entre usuarios.

## 8.2. Aportes relacionados con el desarrollo de capacidades del investigador

La ejecución de este proyecto exige y desarrolla competencias profesionales específicas de la disciplina, propias de la ingeniería telemática y de sistemas. El despliegue y la configuración del clúster de Kubernetes sobre las estaciones de trabajo del laboratorio, la integración de KubeRay y del NVIDIA GPU Operator, y la definición de reglas de selección de nodos y aislamiento de cargas de trabajo desarrollan competencias de orquestación de contenedores y cómputo distribuido en topologías heterogéneas de CPU y GPU. La integración de motores de inferencia empaquetados como imágenes de contenedor, elegidos por medición y con la imagen oficial de *llama.cpp* como candidato preferente para servir modelos ya cuantizados en formato GGUF dentro de la memo-

ria de video disponible, desarrolla, adicionalmente, competencias de ingeniería de plataformas de inferencia bajo restricciones de *hardware*.

La implementación del sistema de reserva de cupos con prioridad por rol, acoplado a los atributos de rol expuestos por el servicio de identidad institucional SAAMFI, exige competencias de diseño de esquemas de gobernanza y de integración con sistemas de control de acceso basado en roles (RBAC), un dominio propio de la ingeniería de *software* orientada a sistemas multiusuario. La construcción del módulo de observabilidad, apoyado en Prometheus, Thanos, Grafana, Loki y el exportador DCGM de NVIDIA, desarrolla competencias de ingeniería de observabilidad de infraestructura distribuida, entendida como la capacidad de instrumentar, recolectar y visualizar métricas de *hardware* e inferencia en tiempo real. Por último, la evaluación de la plataforma mediante pruebas de carga sobre los nodos de inferencia, ejecutadas con *scripts* de Python, desarrolla competencias de diseño y ejecución de evaluación empírica de sistemas de *software*, indispensables para sustentar decisiones de ingeniería con evidencia.

## 8.3. Resultados y entregables

De acuerdo con el formato institucional del anteproyecto, cada resultado esperado y su entregable correspondiente deben trazarse a al menos uno de los objetivos específicos formulados en el Capítulo 3. La Tabla 8.1 presenta esta trazabilidad para los cuatro objetivos específicos del proyecto.

**Tabla 8.1:** *Trazabilidad entre objetivos específicos, resultados esperados y entregables.*

| **Objetivo específico** | **Resultado esperado** | **Entregable** |
|---|---|---|
| Objetivo específico 1: desarrollo del módulo de observabilidad | Panel de telemetría que muestra, por nodo, métricas de *hardware* (temperatura, VRAM, consumo energético) y de carga de trabajo (latencia, *tokens* por segundo, saturación de concurrencia), mediante visualizaciones gráficas interactivas | Módulo de observabilidad integrado a la plataforma (Prometheus, Thanos, DCGM Exporter, Grafana, Loki) y su documentación de despliegue |
| Objetivo específico 2: implementación del sistema de cuotas por rol | Sistema de cuotas y restricciones de infraestructura que reserva cupos de cómputo para las cargas de inferencia, con prioridad por rol, acoplado a los roles provistos por SAAMFI | Módulo administrativo de gestión de cuotas integrado a la plataforma web y manual de administración de políticas de cuotas |

| **Objetivo específico** | **Resultado esperado** | **Entregable** |
|---|---|---|
| Objetivo específico 3: implementación del despliegue de modelos | Clúster de cómputo que ejecuta motores de inferencia intercambiables, empaquetados como imágenes de contenedor, sobre el cual un usuario despliega modelos ya entrenados de forma concurrente desde la plataforma | Clúster desplegado con su configuración versionada, interfaz de despliegue de modelos integrada a la plataforma y manual de despliegue |
| Objetivo específico 4: evaluación de desempeño | Reporte de desempeño de la plataforma bajo pruebas de carga con usuarios concurrentes sobre los nodos de inferencia, incluidos los modelos que no son de lenguaje y el crecimiento de la capacidad por nodo | Informe de resultados de pruebas de carga, con los datos generados por los *scripts* de prueba de carga |

Los cuatro entregables específicos de la Tabla 8.1 convergen en los dos entregables globales exigidos para el proyecto de grado. El primero es el *software* funcional y empaquetado, es decir, el código fuente de la plataforma, desplegado sobre la infraestructura propia del IAsLab y estructurado bajo estándares que permitan su registro como producto tecnológico con derechos de autor. El segundo es la documentación técnica y de usuario, compuesta por los manuales de despliegue de modelos, de visualización de telemetría y de administración de políticas de cuotas que se derivan de cada uno de los módulos anteriores. Ambos entregables globales dependen del cumplimiento conjunto de los cuatro objetivos específicos, porque ningún módulo constituye, por sí solo, la plataforma que exige el objeto del proyecto.

# 9. Avances del proyecto

Este capítulo informa el trabajo realizado hasta el 7 de noviembre de 2026 en la prueba de concepto (POC) del proyecto. El prototipo corre en un único servidor físico del laboratorio, sobre el que se simula el clúster con ocho máquinas virtuales, tres de control, tres de servicios y dos de cómputo sin unidad de procesamiento gráfico (GPU), y cada modelo se sirve en un solo nodo de cómputo. El clúster final también tendrá tres nodos de control, alojados en equipos físicos del laboratorio, de modo que el prototipo reproduce esa topología con máquinas virtuales. El POC permite validar el diseño antes de desplegarlo en la sala 104M en diciembre de 2026, y por esa razón su implementación puede cambiar. Las decisiones que aquí se describen son las vigentes a la fecha de corte.

**[TODO: Describan desde el inicio el equipo con el que trabajan, con su procesador (modelo y núcleos), memoria RAM, almacenamiento y GPU, si la tiene. Hoy solo se menciona la memoria de 60 GiB en la segunda etapa, cuando el lector ya necesitaba ese dato. Acompañen la descripción con una fotografía de la estación o torre. Después pueden recordarla, pero el lector debe conocerla desde el principio.]**

**[TODO: Este capítulo servirá de insumo para el PDG2. Si lo redactan con detalle y precisión, podrá reutilizarse casi directamente en el informe final. Escríbanlo pensando en ese propósito.]**

El trabajo del prototipo se desarrolló en dos etapas, entre el 17 de octubre y el 7 de noviembre de 2026, dentro de los *sprints* 2 y 3 del cronograma de la Figura 7.1. Para validar el diseño de extremo a extremo, el prototipo construyó versiones mínimas de componentes que el cronograma completa en fases posteriores, sin alterar el orden de las fases ni su duración. El *hardware* disponible condicionó el montaje. Sin GPU en el servidor, los motores de inferencia se ejecutan en CPU, lo que limita las mediciones de desempeño a una referencia preliminar, y la validación sobre las GPU de la sala queda para la etapa de despliegue.

## 9.1. Primera etapa: cimientos del clúster

La primera etapa, del 17 al 26 de octubre de 2026, levantó la base sobre la que se construyen los demás componentes. Se montó el servidor y las ocho máquinas virtuales, y sobre ellas se instaló un clúster de Kubernetes con tres nodos de control y una dirección virtual que se mantiene aunque caiga uno de ellos. Sobre ese clúster se desplegaron la entrada fija de la plataforma, la entrega continua desde el repositorio de configuración (GitOps), el cifrado de secretos, el almacenamiento persistente, la base de datos, el registro de imágenes y la observabilidad de métricas y registros. También se activaron las políticas de admisión y el gestor de colas de cómputo, que retiene las cargas que no tienen capacidad disponible. La Figura 9.2 muestra el prototipo al cierre de esta

etapa.

**[TODO: La arquitectura del POC es muy detallada y la figura es demasiado grande, con mucho texto, lo que dificulta su lectura. Presenten primero la arquitectura completa y luego un recorte o ampliación por cada etapa o módulo, de modo que se explique un componente a la vez. Por ejemplo, en el aprovisionamiento muestren con un elemento gráfico qué partes de Kubernetes configuraron y cuáles no.]**

**[TODO: Cuando mencionen una figura, como la Figura 9.2, procuren que aparezca antes que el texto que la cita, o muy cerca. Hoy se menciona en la página 33 y aparece en la 38, al final del capítulo, lejos de su explicación, y el lector no puede comparar el texto con la imagen. Revisen esto en todas las figuras.]**

Durante esta etapa aparecieron tres retos. El primero fue que el sistema operativo de las máquinas virtuales no detectaba el disco ni la red, porque la definición de las máquinas no activaba la interfaz de configuración avanzada y energía con la que el sistema descubre sus dispositivos. Se activó esa interfaz en la definición de las máquinas. El segundo fue que todos los nodos tenían marcas de reserva (*taints*), de modo que los trabajos de instalación de los componentes base no encontraban un nodo donde ejecutarse. Se resolvió dejando los nodos de servicios sin esa marca, identificados solo con una etiqueta. El tercero fue la dependencia de un único nodo de control, cuya caída deja al clúster sin acceso a su interfaz de administración. Los equipos del laboratorio se encienden y se apagan sin aviso, y algunos utilizan discos mecánicos, por lo que ninguno cumple por sí solo la condición de permanecer encendido sin reinicios no anunciados. Por ese motivo el plano de control pasó de uno a tres nodos, de modo que la caída de uno no detenga el clúster, decisión que también se aplica al despliegue en la sala.

## 9.2. Segunda etapa: servicio de modelos y gobierno

**[TODO: El proyecto afirma que la plataforma gobierna cualquier carga de IA y aprendizaje automático, pero los avances solo muestran modelos de lenguaje. Aclaren si el diseño y la prueba de concepto contemplan también modelos desarrollados en PyTorch, scikit-learn u otros marcos. Si los contemplan, indiquen cómo se despliegan y, de ser posible, muestren un ejemplo. Si no, expliquen en qué etapa se incorporarán.]**

La segunda etapa, del 26 de octubre al 7 de noviembre de 2026, construyó el servicio de modelos sobre la base anterior. Cada despliegue se describe como un recurso del clúster que indica el modelo y el motor con que se sirve. La elección del motor depende del formato de los pesos, de modo que los modelos en GGUF se sirven con *llama.cpp* y los modelos en *safetensors* con vLLM. Como línea base, un despliegue se ejecuta como un *Deployment* admitido por la cola de cómputo, y se compara con una variante sobre Ray Serve que usa el mismo motor. En esa variante, un adaptador propio mide el tiempo hasta el primer *token* y la latencia entre *tokens* de cada petición.

Desde el punto de vista de quien usa la plataforma, desplegar un modelo tiene dos momentos. Primero, el administrador da de alta el modelo en el catálogo a partir del identificador de un repositorio externo de modelos. La plataforma muestra una vista previa, descarga los pesos, verifica

su integridad y los guarda en el almacén de objetos, y un modelo que está en uso no se puede eliminar. El administrador también otorga a cada usuario el permiso para desplegar. Después, el usuario elige un modelo del catálogo en la interfaz web, selecciona uno de los motores habilitados y, como opción avanzada, el nodo donde quiere ejecutarlo. El estado del despliegue avanza por en cola, admitido, copiando pesos y en ejecución, lo que en el prototipo toma cerca de un minuto, y el modelo queda publicado en una dirección formada por el nombre del usuario y el del despliegue, con una API compatible con OpenAI. Desde la misma interfaz el usuario sigue los registros de su despliegue, lo detiene, lo vuelve a iniciar o lo elimina, mientras el administrador ve y gestiona los despliegues de todos los usuarios.

La Figura 9.1 muestra lo que ocurre internamente en ese recorrido. La API verifica que el despliegue quepa en la cuota del usuario y lo registra como un recurso del clúster. El operador observa ese recurso y genera lo que el motor necesita para ejecutarse, que incluye su carga de trabajo, el volumen de pesos y la ruta de red. La cola de cómputo admite la carga o la mantiene en espera hasta que haya cuota, y el planificador la ubica en un nodo de cómputo. Allí, un contenedor de inicio copia los pesos desde el almacén de objetos al volumen del nodo, salvo que ya estén, y el motor los carga y queda disponible en una ruta de la puerta de entrada. La API y el operador se comunican únicamente a través del servidor del clúster, de modo que una solicitud se conserva aunque el operador se detenga.

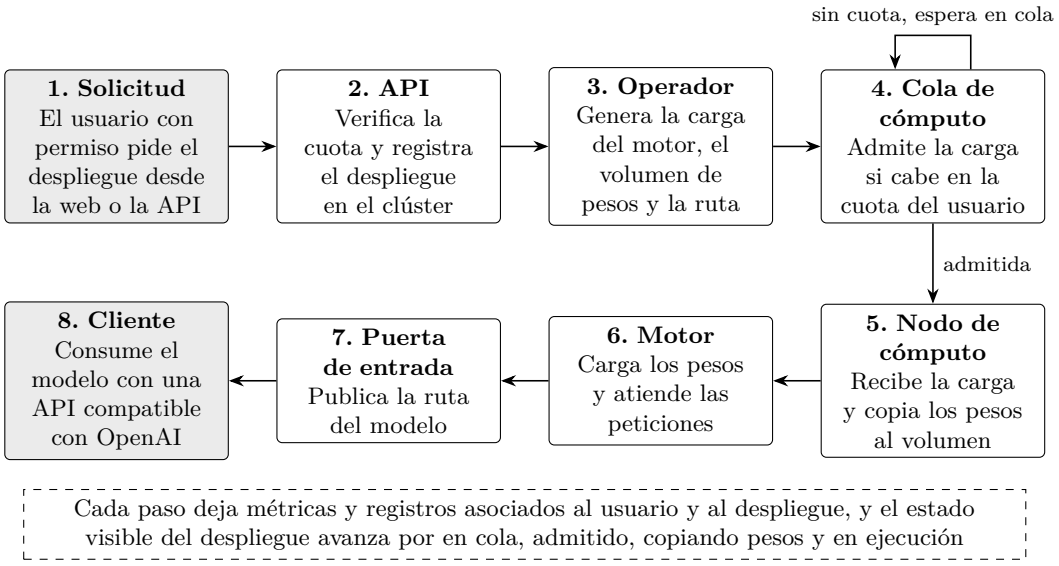



**Figura 9.1:** *Flujo de un despliegue en el POC, desde la solicitud del usuario hasta el consumo del modelo. Las flechas indican el orden de los pasos; el paso 4 retiene la carga en cola mientras el usuario no tenga cuota disponible. Elaboración propia.*

Del lado del gobierno, se implementó el operador que traduce las cuotas de cada usuario en un espacio con su límite, y la cuota de la sala en un fondo común que las cuotas individuales pueden usar. Se verificó en el POC que una solicitud que excede la cuota espera en cola, que una carga puede recuperar cuota cuando otra la necesita, y que la carga expropiada queda identificada en su estado. También se verificó que un agotamiento de memoria en un despliegue no afecta a los vecinos, y que las políticas rechazan los despliegues que no cumplen las etiquetas o los límites de

la sala.

Los retos de esta etapa fueron de recursos. El servidor del POC tiene 60 GiB de memoria, así que cada componente declara la memoria y el procesador que solicita y su límite máximo, de modo que la suma se mantiene dentro de la capacidad del servidor. vLLM agotaba la memoria durante su fase de calentamiento, y se resolvió con su modo de ejecución directa (*eager*), que omite ese calentamiento. El escalador automático de Argo CD no tenía solicitudes de memoria con que calcular su uso, y se fijaron esas solicitudes para que el escalado funcione. Por último, la integración con el proveedor institucional de identidad SAAMFI se encuentra en implementación. Mientras tanto, el prototipo usa un proveedor de identidad local que permite probar el flujo de roles, de autenticación y de permisos de extremo a extremo.

Sobre este servicio se construyeron también el importador de pesos, que verifica el tamaño y la integridad de cada archivo antes de registrarlo, la API con usuarios locales y base de datos, una interfaz web mínima, la lectura de registros filtrada por usuario y un panel de métricas por modelo y por despliegue. Los administradores dan de alta los modelos desde la API o desde la interfaz, y la clave de acceso a repositorios externos de modelos se guarda cifrada.

## 9.3. Mediciones preliminares

La Tabla 9.1 recoge las primeras mediciones del POC. Son referencias de funcionamiento en CPU y no constituyen la evaluación de desempeño del objetivo específico 4, que aún no se ha ejecutado.

**Tabla 9.1:** *Mediciones preliminares del POC (solo CPU; no son una evaluación de desempeño).*

| **Medición** | **Resultado** | **Condición** |
|---|---|---|
| *Tokens* por segundo, Qwen3-0.6B en GGUF Q8_0 con *llama.cpp* | Entre 15 y 17 | 2 CPU por motor |
| *Tokens* por segundo, Qwen3-0.6B con vLLM | Cerca de 3 | 2 CPU |
| Latencia de la variante sobre Ray frente a *llama.cpp* directo | +2,4 % | Misma salida y mismas peticiones |

## 9.4. Reuniones de seguimiento

Las reuniones de seguimiento han sido un insumo constante para el alcance y la arquitectura. En las reuniones con el tutor del 26 de agosto y del 9 de septiembre de 2026 se definió que el núcleo del proyecto es la capa de infraestructura y de servicio, que el sistema de cuotas debe gobernar recursos generales y no solo los *tokens* de los modelos de lenguaje, y que los requerimientos se ordenan por prioridad antes de proponer el corte de alcance. En la elicitación con el profesor Alejandro Muñoz

Bravo, el 4 de septiembre de 2026, se levantaron las necesidades operativas del laboratorio. Con el profesor Juan Carlos Muñoz, quien dirige el macroproyecto, se acordó que las limitaciones de *hardware* permiten desplegar un modelo por equipo, criterio que el POC adopta. Estas decisiones orientan el diseño que se verifica en el prototipo.

## 9.5. Estado al cierre de la etapa

Al cierre de la etapa, el prototipo cubre el ciclo de despliegue de un modelo, desde el alta del modelo hasta su consumo por una API compatible con OpenAI, con cuotas, registros y métricas por usuario. Se encuentran en implementación la integración con SAAMFI, la validación de los componentes sobre las GPU de la sala, las reservas por ventana de tiempo y la prioridad por rol, así como los respaldos fuera del clúster. El despliegue en la sala 104M, previsto para diciembre de 2026, incorporará las GPU del laboratorio y permitirá verificar sobre ellas las mediciones de desempeño del objetivo específico 4.

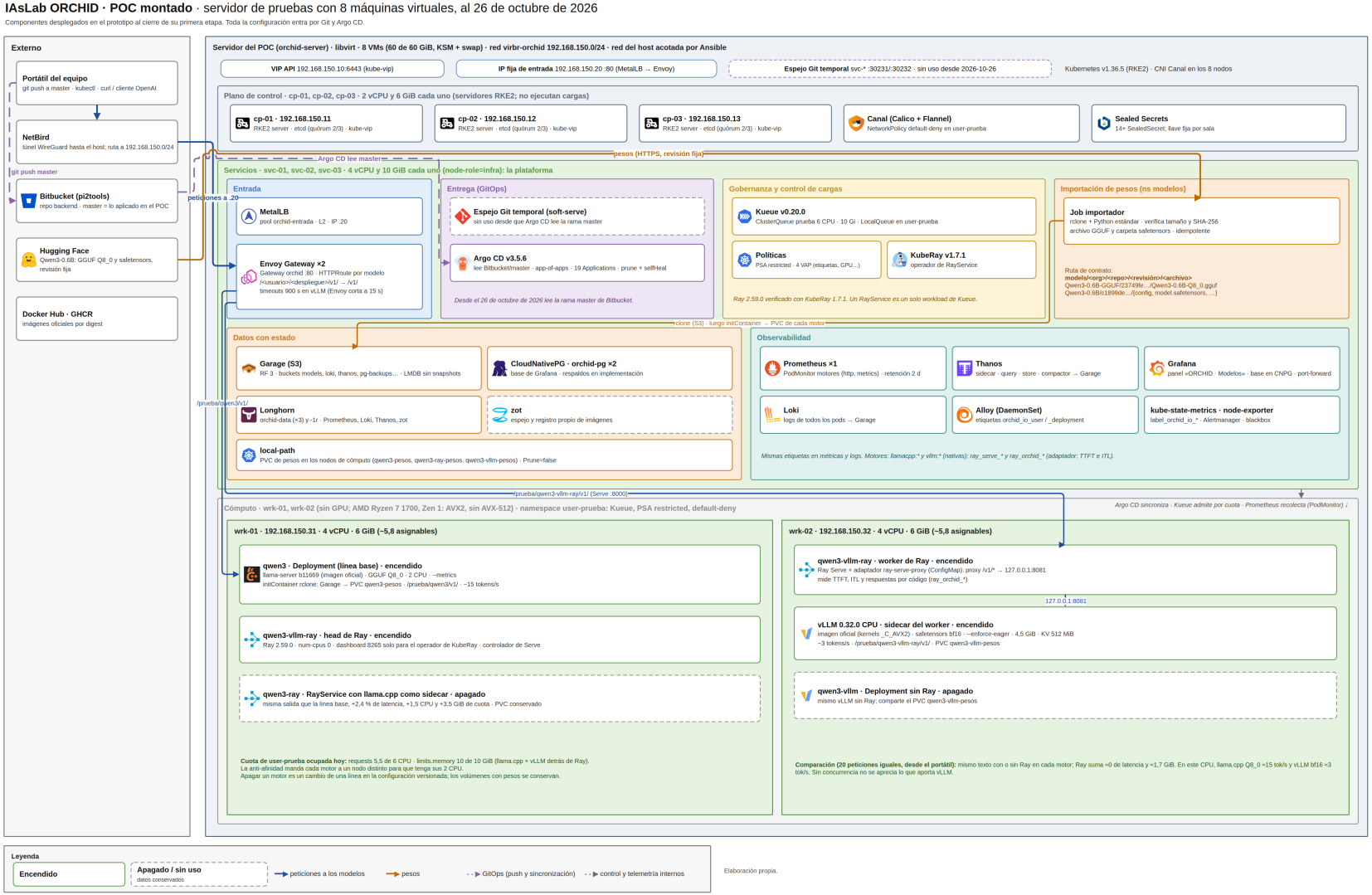



**Figura 9.2:** *Arquitectura del POC al cierre de la primera etapa, el 26 de octubre de 2026. Muestra los tres nodos de control, los tres nodos de servicios con los componentes de la plataforma y los dos nodos de cómputo sin GPU con los motores en ejecución. Las cajas de borde continuo indican componentes encendidos y las de borde discontinuo, componentes apagados o sin uso. Elaboración propia.*

**[TODO: Reconsideren el formato de las referencias. Pueden escoger entre el estilo numérico, con el número entre corchetes, como [1], y el título «Referencias», o el estilo alfanumérico, con un acrónimo del primer autor y el año, como [AWT23]. Elijan uno de los dos y apliquen el formato de forma consistente en todo el documento.]**

# Bibliografía


Ahmed, N., Wahed, M., and Thompson, N. C. (2023). The growing influence of industry in AI research. *Science*, 379(6635):884–886.

Bashir, N., Deng, N., Rzadca, K., Irwin, D. E., Kodak, S., and Jnagal, R. (2021). Take it to the limit: Peak prediction-driven resource overcommitment in datacenters. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*. ACM.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93.

Carrión, C. (2022). Kubernetes as a standard container orchestrator: A bibliometric analysis. *Journal of Grid Computing*, 20:42.

Chen, Y.-D., Zheng, K.-J., Guo, Z.-H., Zhang, Q.-H., Zhang, Y.-H., and Zhai, J.-D. (2026). A survey of quantization in LLM: Unlocking potential hardware efficiency. *Journal of Computer Science and Technology*, 41(1):341–358.

Cohen, M. C., Keller, P. W., Mirrokni, V., and Zadimoghaddam, M. (2019). Overcommitment in cloud services: Bin packing with chance constraints. *Management Science*, 65(7):3255–3271.

Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., and Bianchini, R. (2017). Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167. ACM.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Curran Associates, Inc.

Eken, B., Pallewatta, S., Tran, N. K., Tosun, A., and Babar, M. A. (2026). A multivocal review of MLOps practices, challenges and open issues. *ACM Computing Surveys*, 58(2):1–35.

Fett, D., Küsters, R., and Schmitz, G. (2016). A comprehensive formal security analysis of OAuth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*, pages 1204–1215. ACM.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*.

Gao, Y., He, Y., Li, X., Zhao, B., Lin, H., Liang, Y., Zhong, J., Zhang, H., Wang, J., Zeng, Y., Gui, K., Tong, J., and Yang, M. (2024). An empirical study on low GPU utilization of deep learning jobs. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024)*. ACM.

George, N. (2020). Literature review and implementation overview: High performance computing with graphics processing units for classroom and research use. *arXiv preprint arXiv:2005.07598*.

ggml-org (2026). llama.cpp: LLM inference in C/C++. Documentación en línea (archivo *README*), consultada el 4 de octubre de 2026.

Gu, J., Chowdhury, M., Shin, K. G., Zhu, Y., Jeon, M., Qian, J., Liu, H., and Guo, C. (2019). Tiresias: A GPU cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500. USENIX Association.

Jeon, M., Venkataraman, S., Phanishayee, A., Qian, J., Xiao, W., and Yang, F. (2019). Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association.

Jiang, Z. M. and Hassan, A. E. (2015). A survey on load testing of large-scale software systems. *IEEE Transactions on Software Engineering*, 41(11):1091–1118.

Kreuzberger, D., Kühl, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.

Kubeflow (2026). Profiles and namespaces. Documentación en línea, consultada el 4 de octubre de 2026.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 23)*. ACM.

Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., and Liu, X. (2021). Enjoy your observability: An industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27:25.

Li, H., Wang, S., Tian, X., Wei, W., and Sun, C. (2015). A survey of extended role-based access control in cloud computing. In *Proceedings of the 4th International Conference on Computer Engineering and Networks (CENet 2014)*, volume 355 of *Lecture Notes in Electrical Engineering*. Springer.

Lima, A., Monteiro, L., and Furtado, A. P. (2022). MLOps: Practices, maturity models, roles, tools, and challenges – a systematic literature review. In *Proceedings of the 24th International Conference on Enterprise Information Systems (ICEIS 2022)*, pages 308–320. SciTePress.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of the 7th Conference on Machine Learning and Systems (MLSys 2024)*. Best Paper Award.

LiteLLM (2026a). Budgets, rate limits. Documentación en línea, consultada el 4 de octubre de 2026.

LiteLLM (2026b). Getting started. Documentación en línea, consultada el 4 de octubre de 2026.

Liu, H., Li, Z., Tan, C., Yang, R., Cao, G., Liu, Z., and Guo, C. (2023). Predicting GPU failures with high precision under deep learning workloads. In *Proceedings of the 16th ACM International Systems and Storage Conference (SYSTOR '23)*, Haifa, Israel. ACM.

Mahajan, K., Balasubramanian, A., Singhvi, A., Venkataraman, S., Akella, A., Phanishayee, A., and Chawla, S. (2020). Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Jin, H., Chen, T., and Jia, Z. (2025). Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37.

Montesi, F. and Weber, J. (2016). Circuit breakers, discovery, and API gateways in microservices. Preprint arXiv:1609.05830.

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association.

Narayanan, D., Santhanam, K., Kazhamiaka, F., Phanishayee, A., and Zaharia, M. (2020). Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association.

Qiao, A., Choe, S. K., Subramanya, S. J., Neiswanger, W., Ho, Q., Zhang, H., Ganger, G. R., and Xing, E. P. (2021). Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association.

SchedMD (2026). Slurm workload manager: Overview. Documentación en línea, consultada el 4 de octubre de 2026.

Schwaber, K. and Sutherland, J. (2020). The scrum guide: The definitive guide to scrum: The rules of the game.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*. Curran Associates, Inc.

Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., and Villalobos, P. (2022). Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE.

Shamim, S. I., Gibson, J. A., Morrison, P., and Rahman, A. (2022). Benefits, challenges, and research topics: A multi-vocal literature review of kubernetes. Preprint arXiv:2211.07032.

Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., and Wilkes, J. (2015). Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys 15)*. ACM.

Weitzel, D., Graves, A., Albin, S., Zhu, H., Würthwein, F., Tatineni, M., Mishin, D., Graham, J., Khoda, E. E., Sada, M. F., Smarr, L., and DeFanti, T. (2025). The national research platform: Stretched, multi-tenant, scientific Kubernetes cluster. In *Practice and Experience in Advanced Research Computing (PEARC '25)*. ACM. Preprint arXiv:2505.22864.

Weng, Q., Xiao, W., Yu, Y., Wang, W., Wang, C., He, J., Li, Y., Zhang, L., Lin, W., and Ding, Y. (2022). MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960. USENIX Association.

Xiao, W., Bhardwaj, R., Ramjee, R., Sivathanu, M., Kwatra, N., Han, Z., Patel, P., Peng, X., Zhao, H., Zhang, Q., Yang, F., and Zhou, L. (2018). Gandiva: Introspective cluster scheduling for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association.

Xu, K., Sun, D., Wang, H., Ren, Z., Wan, X., Liao, X., Wang, Z., Zhang, J., and Chen, K. (2025). Design and operation of shared machine learning clusters on campus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2025), Volume 1*. ACM. Preprint available as arXiv:2110.01556.

Xu, Q., Gao, Y., and Wei, J. (2024). An empirical study on kubernetes operator bugs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1746–1758.

Ye, Z., Gao, W., Hu, Q., Sun, P., Wang, X., Luo, Y., Zhang, T., and Wen, Y. (2024). Deep learning workload scheduling in GPU datacenters: A survey. *ACM Computing Surveys*, 56(6):146:1–146:38.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. (2022). Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association.

# A. Análisis de participación

Este anexo identifica a los interesados del proyecto, los clasifica según su poder para influir en las decisiones y su interés en la plataforma, y analiza los efectos éticos y de impacto sobre los más afectados. Para ello aplica el mapeo de interesados con la matriz poder-interés, que ubica a cada actor en uno de cuatro cuadrantes. El poder expresa la capacidad de un actor para influir en las decisiones del proyecto, y el interés expresa el grado en que la plataforma lo afecta.

## A.1. Identificación de los interesados

El proyecto desarrolla una plataforma de *software* para una infraestructura de cómputo universitaria, en un contexto académico, y los interesados se derivan de los actores descritos en el Capítulo 1 y en el Capítulo 2. La Tabla A.1 los agrupa en las categorías de usuarios finales, cliente, equipo técnico, institución, actores externos y actores potencialmente impactados.

No se identificaron actores regulatorios ni legales con influencia directa sobre las decisiones del proyecto, de modo que esa categoría no aparece en la tabla.

**Tabla A.1:** *Interesados del proyecto, su categoría y su relación con la plataforma.*

| **Interesado** | **Categoría** | **Relación con el proyecto** |
|---|---|---|
| Estudiantes y profesores de la universidad | Usuarios finales | Requieren GPU para sus proyectos e investigaciones, se identifican por sus roles institucionales y son quienes consumirán el servicio de despliegue de modelos. |
| Usuarios del plano de control de entrenamiento | Usuarios finales | Entrenan modelos con el sistema web de TRAINI sobre las mismas estaciones de trabajo que usará la plataforma. |
| Personal administrador del IAsLab | Cliente | Opera los equipos, aporta los requerimientos del laboratorio y asumirá el soporte y el mantenimiento de la plataforma cuando termine el proyecto. |
| Autores del proyecto | Equipo técnico | Diseñan, construyen y evalúan la plataforma. |
| Tutor del proyecto | Equipo técnico | Orienta el desarrollo del proyecto y responde académicamente por él. |
| Universidad Icesi | Organización o institución | Es propietaria de la infraestructura en la que invirtió y cuyo aprovechamiento busca mejorar el proyecto. |

| Interesado | Categoría | Relación con el proyecto |
|---|---|---|
| Servicio institucional de identidad (SAAMFI) | Actores externos | Provee las identidades y los roles de los que dependen el control de acceso y la gobernanza de cuotas. |
| Docentes y estudiantes de las clases que usan la sala 104M | Actores potencialmente impactados | Usan las mismas estaciones para actividades académicas, que deben seguir siendo utilizables mientras el clúster opera. |

## A.2. Clasificación en la matriz poder-interés

La Tabla A.2 estima el poder y el interés de cada interesado y le asigna la estrategia que corresponde a su cuadrante, y la Figura A.1 presenta el resultado en la matriz. Los interesados de alto poder y alto interés, y los de bajo poder y alto interés, se consideran críticos y son los que se analizan en la sección siguiente.

**Tabla A.2:** *Poder, interés y estrategia asignada a cada interesado.*

| Interesado | Poder | Interés | Estrategia | Justificación |
|---|---|---|---|---|
| Autores del proyecto | Alto | Alto | Gestionar de cerca | Toman las decisiones de diseño y construcción, y responden por el resultado. |
| Tutor del proyecto | Alto | Alto | Gestionar de cerca | Orienta el desarrollo del proyecto y responde académicamente por él. |
| Personal administrador del IAsLab | Alto | Alto | Gestionar de cerca | Define los requerimientos operativos, opera los equipos y asumirá la operación posterior. |
| Universidad Icesi | Alto | Bajo | Mantener satisfecho | Decide sobre la inversión en equipos, pero el proyecto afecta su operación solo de forma indirecta. |
| Servicio institucional de identidad (SAAMFI) | Alto | Bajo | Mantener satisfecho | Sus cambios o su caída condicionan el control de acceso y las cuotas, pero el servicio no depende del proyecto. |
| Estudiantes y profesores de la universidad | Bajo | Alto | Mantener informado | Usan la plataforma y sus cupos los afectan directamente, pero no deciden sobre su diseño. |
| Usuarios del plano de control de entrenamiento | Bajo | Alto | Mantener informado | Comparten el mismo *hardware*, por lo que cualquier regla de reparto de las GPU los afecta. |

| Interesado | Poder | Interés | Estrategia | Justificación |
|---|---|---|---|---|
| Docentes y estudiantes de las clases de la sala 104M | Bajo | Alto | Mantener informado | Sus clases comparten las estaciones que usa la plataforma, y esta debe dejarlas utilizables. |

Ningún interesado identificado queda en el cuadrante de bajo poder y bajo interés, por lo que ese cuadrante permanece vacío en la Figura A.1.

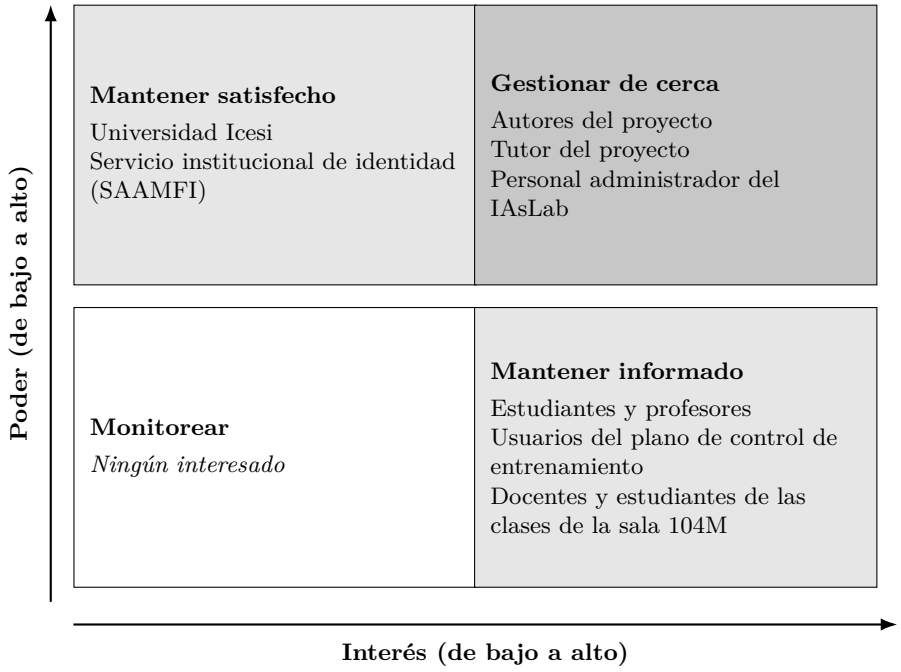



**Figura A.1:** *Matriz poder-interés de los interesados del proyecto. Elaboración propia a partir de la Tabla A.2.*

## A.3. Análisis ético y de impacto

Los interesados críticos son los autores, el tutor, el personal administrador del IAsLab, los estudiantes y profesores, los usuarios del plano de control de entrenamiento y los docentes y estudiantes de las clases de la sala 104M. El análisis de sus riesgos se organiza en los tres aspectos que corresponden a todo proyecto, seguidos de dos que se aplican porque la plataforma registra el uso individual y toma decisiones automáticas sobre los recursos.

**Riesgos de exclusión o mala usabilidad** La plataforma autentica a los usuarios contra el servicio institucional de identidad, de modo que queda fuera del servicio quien no cuente con una identidad institucional vigente. Entre quienes sí acceden, las cuotas y las prioridades por

rol pueden dejar sin cupo o interrumpir la carga de un usuario de menor prioridad cuando no hay capacidad libre. Este riesgo se atenúa porque la plataforma explica al usuario qué límite alcanzó cuando rechaza o encola su solicitud, y le muestra su cuota asignada y su consumo actual. Además, la plataforma se diseña para que un estudiante la opere sin conocimientos de línea de comandos ni de Kubernetes, de modo que no reproduzca la barrera de acceso que motivó el proyecto.

**Impactos organizacionales** La plataforma sustituye la entrega manual de credenciales y la atención individual de cada solicitud, ya que todo usuario autenticado recibe un rol por defecto, sin que el administrador tenga que crear cuentas ni credenciales por persona, lo que reduce la carga operativa de los administradores. El administrador conserva dos decisiones, que son otorgar el permiso para desplegar modelos y dar de alta los modelos de lenguaje en la plataforma, con el fin de proteger los recursos compartidos de las salas. A cambio, el soporte y el mantenimiento de la plataforma pasan al personal administrador del laboratorio cuando termina el proyecto. Para facilitar esa transición, la configuración del clúster se versiona y se automatiza, de modo que el laboratorio pueda redesplegarla en otra sala sin reconstruirla.

**Dependencia tecnológica** El control de acceso y las cuotas por rol dependen del servicio institucional de identidad, de modo que un cambio en su interfaz o una caída afecta el uso de la plataforma. Además, la plataforma se diseña solo para nodos con aceleradores de un único fabricante, y desplegarla sobre equipos de otro proveedor exigiría sustituir las capas de aprovisionamiento y de telemetría. El análisis de riesgos del Anexo E recoge ambas dependencias, y la plataforma aísla la traducción de los roles del servicio de identidad en un componente propio, de modo que un cambio en ese servicio obligue a ajustar solo ese componente.

**Riesgos por uso de datos** La plataforma asocia las métricas de carga al usuario y al despliegue que las originó, de modo que genera datos de uso individual. El panel de consumo por usuario está destinado al administrador, mientras que cada usuario ve su propia cuota y su consumo. El procesamiento de inferencia ocurre en servidores propios de la universidad, por lo que los datos de las consultas no salen a plataformas de terceros.

**Decisiones automatizadas que afectan a las personas** La plataforma rechaza o encola las solicitudes que exceden una cuota y expropia los recursos de una carga de menor prioridad cuando una solicitud de rol prioritario lo requiere. Estas decisiones afectan el trabajo de los usuarios, y se atenúan porque cada rol tiene un nivel de prioridad declarado, de modo que la contención de recursos se resuelve por una regla conocida en lugar de por orden de llegada.

# B. Árbol de problemas

Este anexo presenta el árbol de problemas que respalda la descripción del problema del Capítulo 2. El árbol reúne en una sola vista la causa principal, los factores causales, el problema central y los efectos que se derivan de él, de modo que el lector pueda comprobar la jerarquía que el capítulo describe en prosa.

La Figura B.1 se lee de abajo hacia arriba, y las flechas indican la dirección de la relación causal. En la base se ubica la causa principal, de la que se desprenden tres factores causales que, en conjunto, originan el problema central. De ese problema se derivan dos efectos principales, que a su vez producen efectos secundarios agrupados según quién los experimenta, ya sea la comunidad académica, el personal administrador o la propia infraestructura.

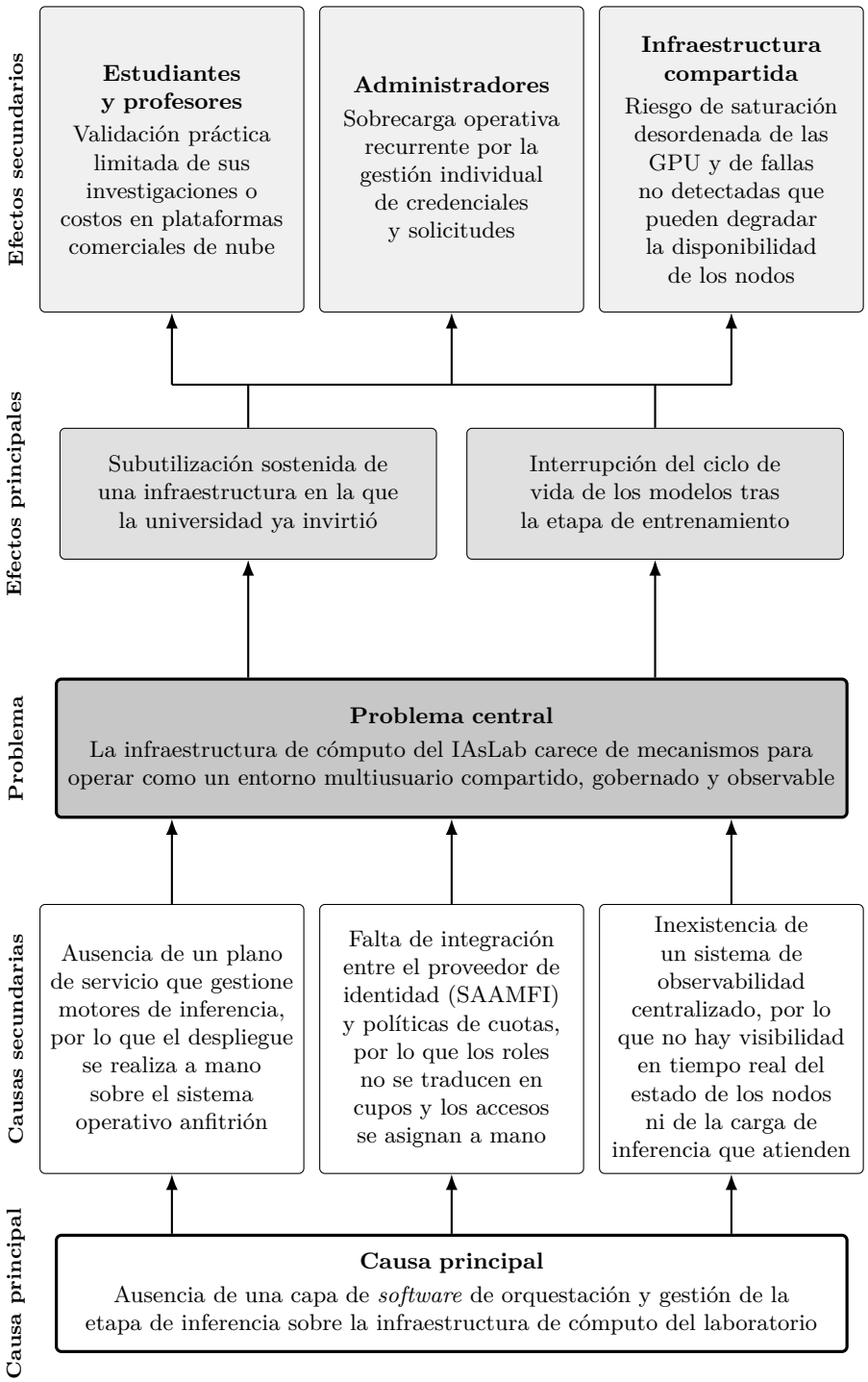



**Figura B.1:** *Árbol de problemas del proyecto. Las flechas indican la relación causal y el diagrama se lee de abajo hacia arriba. Elaboración propia a partir del Capítulo 2.*

# C. Árbol de objetivos

Este anexo presenta el árbol de objetivos que respalda la formulación de los objetivos del Capítulo 3. Se obtuvo al plantear en positivo cada elemento del árbol de problemas del Anexo B, de modo que cada factor causal se convierte en un medio, el problema central en el objetivo central y los efectos en fines.

La Figura C.1 se lee de abajo hacia arriba. Cada medio conduce a uno de los tres objetivos específicos de construcción, y el conjunto de medios conduce al objetivo general, del que se derivan los fines principales y los fines secundarios. Las columnas conservan el orden del Anexo B, de modo que cada medio queda frente a la causa que atiende. Los textos de los objetivos específicos dentro de las cajas están resumidos, y su redacción completa se encuentra en la Sección 3.2.

El objetivo específico 4 no aparece en el árbol porque ninguna causa del árbol de problemas se asocia a él. Ese objetivo verifica el desempeño de la plataforma una vez construida.

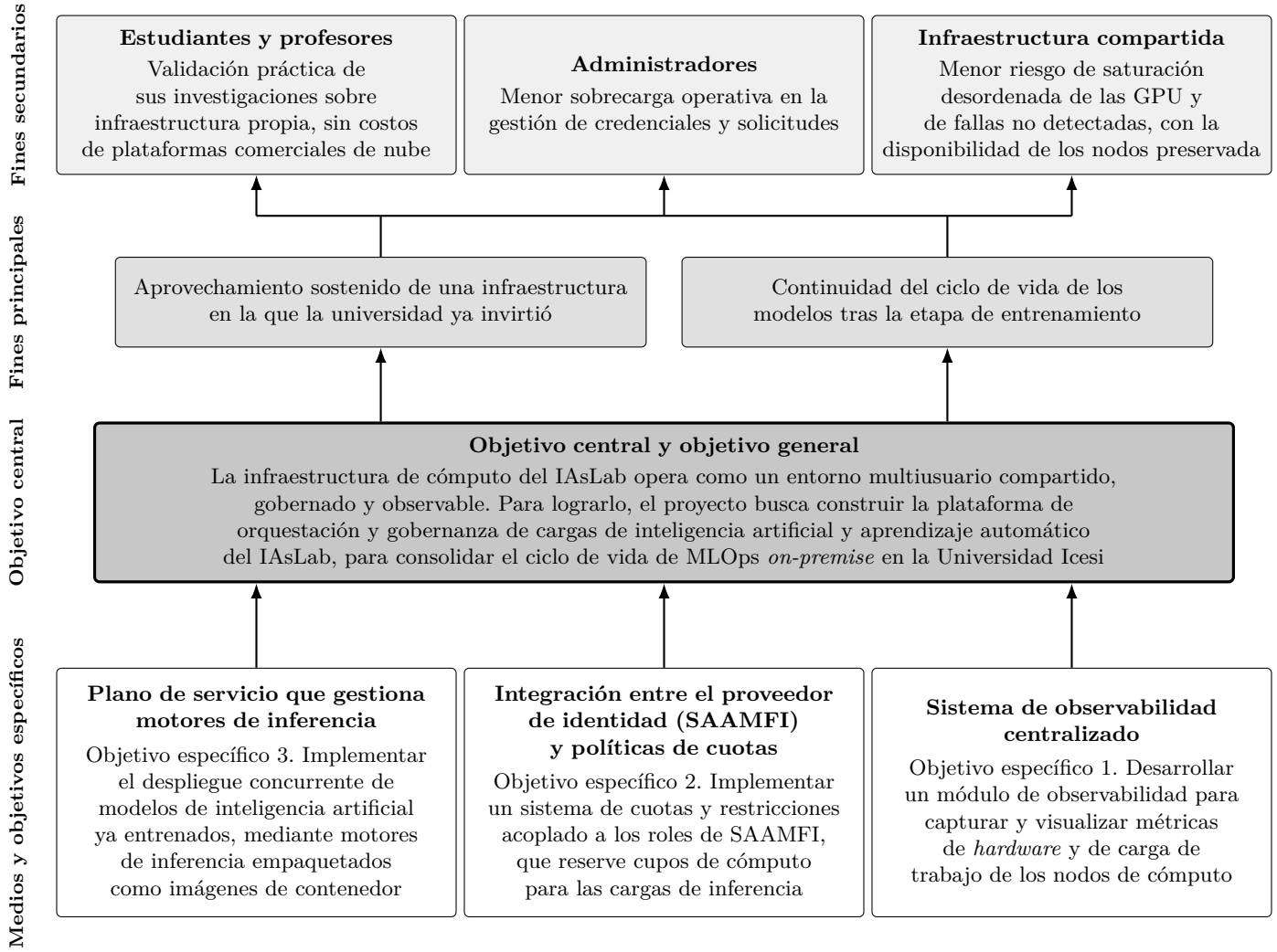



**Figura C.1:** *Árbol de objetivos del proyecto. Las flechas indican que cada medio conduce al objetivo central y que este conduce a los fines, de modo que el diagrama se lee de abajo hacia arriba. Elaboración propia a partir del Anexo B y del Capítulo 3.*

# D. Requerimientos del sistema por objetivo

Este anexo reúne los requerimientos del proyecto, agrupados por el objetivo específico que cada uno sustenta, y cada uno lleva un código de la forma Rk-nn, donde k señala el objetivo y nn el consecutivo dentro de él. Se levantaron a partir de las sesiones de elicitación con el personal del IAsLab, que sirvieron como insumo. Después de revisarlos y ajustarlos, se cruzaron con los objetivos específicos para validar y definir el alcance propuesto del proyecto. Con ese alcance fijado, el equipo los refinó y priorizó, y el tutor los revisó. La agrupación resultante bajo cada objetivo específico permite que un evaluador verifique su cumplimiento con los entregables del Capítulo 8. Las Secciones D.1 a D.4 presentan el conjunto en los mismos subsistemas funcionales con los que se especificó el proyecto, en el orden de los objetivos específicos del Capítulo 3.

## D.1. Módulo de observabilidad (objetivo específico 1)

Estos requerimientos permiten verificar el cumplimiento del objetivo específico 1.

**Infraestructura base de telemetría**

**R1-01** Despliega Prometheus dentro del clúster como servicio permanente que centraliza la recolección de métricas de todos sus componentes.

**R1-02** Recolecta las métricas nativas de Kubernetes de nodos, *pods* y despliegues mediante kube-state-metrics y node-exporter, sin instrumentación adicional.

**R1-03** Despliega Thanos sobre Prometheus, con las series temporales conservadas a largo plazo y consultables desde un punto único, sin que la retención dependa del disco de una sola instancia.

**R1-04** Despliega Loki junto con un agente de recolección en cada nodo que centraliza la salida estándar y de error de todos los *pods* del clúster.

**R1-05** Despliega Grafana con Prometheus, Thanos y Loki registrados como fuentes de datos, consultables desde una sola interfaz.

**R1-06** Permite configurar la ventana de retención de métricas y registros.

**R1-07** Etiqueta métricas y registros con las mismas etiquetas de nodo, espacio de nombres, *pod* y despliegue.

**R1-08** Refleja la telemetría con una latencia de actualización suficiente para detectar una condición de riesgo mientras aún puede evitarse.

**Telemetría de *hardware***

**R1-09** Ejecuta el exportador DCGM de NVIDIA como *DaemonSet* en los nodos con GPU y expone sus métricas a Prometheus.

**R1-10** Recolecta la temperatura del núcleo y de la memoria de cada GPU.

**R1-11** Recolecta la utilización de VRAM de cada GPU, en valor absoluto y en porcentaje.

**R1-12** Recolecta el consumo de potencia en vatios de cada GPU.

**R1-13** Recolecta los estados de limitación térmica y de potencia (*throttling*) de cada GPU.

**R1-14** Recolecta el *throughput* del bus PCIe de cada GPU.

**R1-15** Recolecta el uso del procesador (CPU, *Central Processing Unit*), la memoria de acceso aleatorio (RAM, *Random Access Memory*) y la entrada y salida (E/S) de disco de cada nodo.

**R1-16** Recolecta el espacio disponible en la partición dedicada a pesos de modelos.

**R1-17** Recolecta el estado de alcanzabilidad de cada nodo, y distingue un equipo apagado de uno encendido que dejó de responder.

**R1-18** Captura la telemetría de *hardware* de un nodo sin importar qué carga esté corriendo en él.

#### Telemetría de la carga servida

**R1-19** Expone un punto `/metrics` en cada *pod* de motor de inferencia y lo registra como objetivo de recolección de Prometheus.

**R1-20** Mide el tiempo hasta el primer *token* por despliegue y por modelo.

**R1-21** Mide la latencia entre *tokens* consecutivos por despliegue.

**R1-22** Mide los *tokens* por segundo de entrada y de salida por despliegue.

**R1-23** Mide la cantidad de peticiones en cola y el nivel de saturación de concurrencia de cada motor.

**R1-24** Mide la latencia de petición y la tasa de peticiones atendidas de las cargas de inteligencia artificial que no son de lenguaje, sin depender de métricas de *tokens*.

**R1-25** Mide la tasa de errores y de peticiones rechazadas por despliegue.

**R1-26** Asocia cada métrica de carga al usuario y al despliegue que la originó.

#### Visualización y alertas

**R1-27** Ofrece un panel de *hardware* filtrable por nodo, con temperatura, VRAM, potencia, PCIe, CPU y RAM.

**R1-28** Ofrece un panel de carga servida filtrable por modelo y por despliegue, con tiempo al primer *token*, latencia, *throughput* y saturación.

**R1-29** Ofrece un panel de consumo por usuario, accesible sin recurrir a la línea de comandos.

**R1-30** Deja configuradas reglas de alerta sobre cualquiera de las métricas recolectadas, con umbrales ajustables, apoyadas en la capacidad de alertas del conjunto de observabilidad.

**R1-33** Conserva los registros y la telemetría previos a un fallo de nodo.

## D.2. Gobernanza de recursos y cuotas por rol (objetivo específico 2)

Estos requerimientos permiten verificar el cumplimiento del objetivo específico 2. El grupo de identidad y acceso se presenta primero porque la plataforma necesita saber quién solicita el recurso antes de aplicarle una cuota; los grupos de cuotas, admisión y el módulo administrativo que los opera siguen a continuación.

**Identidad y control de acceso**

**R2-01** Autentica a los usuarios contra SAAMFI mediante OAuth 2.0 y OIDC (*OpenID Connect*).

**R2-02** Redirige al usuario autenticado al panel principal de la aplicación.

**R2-03** Traduce los atributos del *token* de SAAMFI a los roles de la plataforma en un componente aislado.

**R2-04** Define un rol de administrador propio, gestionable de forma independiente de SAAMFI.

**R2-05** Asigna el rol de usuario por defecto a toda identidad autenticada que no sea administradora.

**R2-06** Deriva del rol de la plataforma los permisos con los que el usuario opera sobre el clúster.

**R2-07** Registra quién desplegó, detuvo o modificó cada recurso y cada cuota.

**Cuotas y aislamiento**

**R2-08** Cuenta con un mecanismo de cuotas que acota el consumo de CPU, memoria y GPU de cada usuario.

**R2-09** Aísla los recursos de cada usuario, de modo que la carga de uno no interfiere con el desempeño ni con los despliegues de otro.

**R2-10** Acota también el consumo de un despliegue individual.

**R2-11** Rechaza o encola la solicitud que excede la cuota del usuario, con un mensaje que explica cuál límite se alcanzó.

**R2-12** Define las cuotas como plantillas por rol, con la posibilidad de sobrescribirlas para un usuario concreto.

**R2-13** Reserva CPU y memoria para los componentes de la propia plataforma y deja la GPU para los motores de inferencia.

**R2-14** Permite una sobreasignación configurable entre el 10 % y el 20 % sobre la capacidad nominal.

**R2-15** Impide que la sobreasignación lleve un nodo más allá de su capacidad física real.

**R2-16** Garantiza que la falla o el congelamiento del contenedor de un usuario no desestabilice los contenedores vecinos ni el nodo anfitrión.

#### Admisión, prioridad y reservas

**R2-17** Encola la solicitud que no encuentra capacidad disponible, en lugar de descartarla.

**R2-19** Asocia un nivel de prioridad a cada rol.

**R2-20** Permite que una solicitud de rol prioritario expropie los recursos de una carga de menor prioridad cuando no haya capacidad libre.

**R2-21** Avisa al usuario cuya carga fue expropiada y conserva sus registros de sesión.

**R2-22** Permite reservar cupos de GPU por adelantado para una ventana de tiempo definida.

**R2-23** Inicia y termina la reserva de forma automática en las horas declaradas.

**R2-24** Avisa al usuario antes del vencimiento de su reserva.

**R2-25** Guarda los registros y el estado de la sesión al vencer la reserva, antes de liberar o congelar el contenedor.

**R2-26** Libera los recursos de una reserva vencida.

**R2-27** Rechaza la reserva que exceda la capacidad disponible de la sala en esa ventana.

**R2-28** Expone una API con operaciones de creación, consulta, modificación y borrado sobre cuotas y reservas.

**R2-29** Aplica las cuotas, la prioridad y la contabilidad de consumo a cualquier carga de inteligencia artificial, sea o no un modelo de lenguaje.

#### Módulo administrativo

**R2-30** Ofrece una interfaz administrativa web que cubre toda la operación de gobernanza.

**R2-31** Permite al administrador modificar los límites de cuota de un rol o de un usuario y aplicar el cambio al clúster sin reiniciar nada.

**R2-32** Permite al administrador asignar y retirar el rol de administrador a otra identidad.

**R2-33** Muestra al administrador todos los despliegues en ejecución, sin importar de qué usuario sean.

**R2-34** Permite al administrador encender, apagar o deshabilitar cualquier despliegue.

**R2-35** Muestra al administrador el consumo de cuota de cada usuario en un periodo consultable.

**R2-36** Muestra al administrador la capacidad libre y comprometida por nodo.

## D.3. Despliegue de modelos y ejecución de cargas de IA (objetivo específico 3)

Estos requerimientos permiten verificar el cumplimiento del objetivo específico 3, en el orden en que el clúster necesita quedar operable, desde su base hasta la interfaz con la que un usuario lo consume, pasando por la orquestación de cargas y los motores que soporta.

**Clúster base**

**R3-01** Corre sobre un clúster de Kubernetes instalado en las estaciones de trabajo de un laboratorio de cómputo, con tres nodos de control y el resto como nodos de trabajo, desplegado inicialmente en la sala 104M y extensible a otras salas del laboratorio.

**R3-02** Aloja los tres nodos de control en máquinas que permanecen encendidas sin reinicios no anunciados, de modo que la caída de uno no detiene el plano de control.

**R3-03** Configura la red del clúster mediante una interfaz de red de contenedores (CNI, *Container Network Interface*) que permite la comunicación entre los *pods* de distintos nodos según la política de red definida.

**R3-04** Despliega el NVIDIA GPU Operator para aprovisionar controladores, *runtime* y complemento de dispositivo de forma declarativa, de modo que los nodos expongan el recurso `nvidia.com/gpu`.

**R3-05** Dirige las cargas aceleradas únicamente a los nodos con GPU mediante selectores de nodo, marcas y tolerancias.

**R3-06** Etiqueta cada nodo con sus capacidades de *hardware*.

**R3-07** Permite incorporar o retirar un nodo del clúster sin reinstalar los demás.

**R3-08** Limita los recursos que toma de cada estación, de modo que los equipos sigan siendo utilizables en las clases de la sala.

**R3-09** Permite marcar un nodo como no planificable y desalojar sus cargas de forma ordenada.

**R3-10** Mantiene toda la configuración del clúster versionada en un repositorio y aplicable de forma automatizada.

**R3-11** Funciona por completo sobre la infraestructura del laboratorio, sin depender de servicios de nube pública.

**R3-12** Almacena los pesos de los modelos en una partición de disco dedicada, separada del disco raíz y de los registros del sistema.

**R3-13** Expone esa partición al clúster como volumen persistente reclamable por los despliegues.

**R3-14** Conserva los pesos descargados entre reinicios de *pod*.

#### Orquestación de cargas

**R3-15** Declara cada servicio de inferencia como un recurso del clúster gestionado por un controlador.

**R3-16** Traduce la petición de despliegue de un usuario en la declaración que el clúster aplica.

**R3-17** Despliega un modelo tanto dentro de un solo nodo como repartido entre varios cuando la infraestructura de red lo permita, apoyándose en Ray y limitándose a las capacidades que este ofrezca.

**R3-18** Sostiene varios despliegues concurrentes de distintos usuarios sobre el clúster sin degradación significativa dentro de los límites de cuota.

**R3-19** Refleja el estado real de cada despliegue, distinguiendo al menos pendiente, en ejecución, fallido y detenido.

**R3-20** Expone al usuario los registros de su propio despliegue.

**R3-21** Deja el modelo desplegado accesible dentro del clúster para las pruebas de carga del Objetivo específico 4.

#### Motores y modelos

**R3-22** Empaqueta cada motor de inferencia como imagen de contenedor versionada en un registro accesible desde el clúster.

**R3-23** Incluye una imagen oficial de *llama.cpp* empaquetada como contenedor, capaz de ejecutar pesos GGUF cuantizados dentro de los 24 GB de VRAM de una RTX 4090.

**R3-25** Permite añadir un motor de inferencia nuevo sin modificar el resto de la plataforma.

**R3-26** Despliega modelos cuyos pesos ya vienen cuantizados y se apoya en la cuantización que el motor aplica al cargarlos, sin ejecutar un proceso de cuantización propio.

**R3-27** Ejecuta cargas de inteligencia artificial que no son modelos de lenguaje, entregadas como archivo de pesos en un formato que sirve un motor del catálogo, por el mismo camino de despliegue que un modelo de lenguaje.

**R3-28** Rechaza, con un mensaje explicativo, el despliegue de un modelo que no cabe en la memoria de video de un nodo.

**R3-29** Permite dar de alta un modelo nuevo indicando un identificador de repositorio externo o subiendo el archivo de pesos, y solo el administrador da de alta los modelos de lenguaje.

#### Interfaz de despliegue

**R3-31** Ofrece al usuario un catálogo de los modelos ya entrenados disponibles.

**R3-32** Permite al usuario elegir el motor de ejecución entre los habilitados antes de desplegar.

**R3-33** Permite al usuario fijar el nodo de despliegue o dejar la decisión al planificador del clúster.

**R3-34** Permite al usuario iniciar y detener su despliegue desde la interfaz.

**R3-35** Muestra al usuario su cuota asignada y su consumo actual.

**R3-36** Es operable por un estudiante sin conocimientos de línea de comandos ni de Kubernetes.

## D.4. Evaluación de desempeño (objetivo específico 4)

Estos requerimientos permiten verificar el cumplimiento del objetivo específico 4.

**Medición de desempeño**

**R4-01** Cuenta con *scripts* sencillos de Python, externos a la plataforma, que ejecutan escenarios de prueba declarados en un archivo de especificación.

**R4-02** Permite configurar el número de usuarios simultáneos del escenario de carga.

**R4-03** Genera peticiones de inferencia concurrentes contra los despliegues activos desde fuera de la aplicación.

**R4-04** Mide en cada ejecución el tiempo al primer *token*, la latencia entre *tokens*, los *tokens* por segundo de entrada y de salida, el tiempo total de respuesta y la tasa de error.

**R4-05** Correlaciona los resultados de la prueba con la telemetría de *hardware* del mismo periodo.

**R4-06** Registra las métricas de red durante la prueba y descarta explícitamente el ancho de banda de la sala como cuello de botella silencioso.

**R4-07** Permite ejecutar el mismo escenario contra motores de inferencia distintos en condiciones idénticas.

**R4-08** Permite programar la ejecución de las pruebas fuera del horario de clases.

**R4-10** Produce un reporte que compara latencias y *throughput* entre escenarios de carga y entre motores.

**R4-11** Mide, para cargas que no son modelos de lenguaje, la cantidad de solicitudes concurrentes que soporta cada nodo y cómo crece esa capacidad al aumentar el número de nodos conectados.

# E. Análisis de riesgos y limitaciones

El despliegue de una plataforma sobre infraestructura de cómputo compartida conlleva incertidumbres técnicas y de gestión que pueden afectar el alcance, el cronograma o la calidad de la solución. Los riesgos identificados a continuación se derivan de las restricciones operativas y supuestos del proyecto, se concentran en los niveles técnico y metodológico, y se presentan junto con la dimensión que comprometen y las estrategias previstas para su mitigación.

**Falta de disponibilidad de los nodos de cómputo del IAsLab** Compromete el cronograma, dado que las fases de desarrollo y, en particular, la evaluación empírica dependen del acceso continuo a los nodos de cómputo del laboratorio, condición que el proyecto asume vigente pero no controla directamente. Como estrategia de mitigación, las actividades de desarrollo que no requieren *hardware* de inferencia (por ejemplo, la definición del esquema lógico de cuotas) se priorizan en los periodos de menor disponibilidad de los nodos.

**Cambios o caída del servicio SAAMFI** SAAMFI es el proveedor de identidad institucional de la Universidad Icesi, que emite un *token* con los roles y permisos del usuario, y cada sistema que lo integra decide cómo usar esos atributos. Este riesgo compromete el alcance y el cronograma, pues el control de acceso y la gobernanza de cuotas por rol dependen de los atributos que provee ese servicio. La mitigación consiste en aislar la lógica de mapeo de roles en un componente propio de la plataforma, de modo que un cambio en la interfaz de SAAMFI exija ajustar solo ese componente, sin rediseñar el esquema de cuotas completo.

**Saturación de VRAM y congelamiento de nodos GPU** Compromete la calidad del servicio de inferencia y, potencialmente, el cronograma de las pruebas de carga. El equipo del laboratorio ha confirmado que un uso sostenido de la GPU entre 90 % y 95 % congela las máquinas de la sala, y que la causa física es que la fuente de poder de la torre no soporta el consumo eléctrico resultante. Liu et al. (2023) muestran que las fallas de GPU bajo cargas de aprendizaje profundo pueden predecirse con modelos entrenados sobre datos recolectados en producción, y en esa línea la mitigación prevista es que el módulo de observabilidad, formulado en el objetivo específico 1 (Sección 3.2), deje configuradas, sobre el propio conjunto de observabilidad, reglas de alerta ante niveles de saturación críticos de la VRAM, cuyo umbral se calibrará durante la implementación con base en la telemetría observada, y conserve los registros y la telemetría previos a una falla para diagnosticarla después.

**Uso simultáneo de los equipos para actividades académicas** Los equipos de la sala deben seguir siendo usables para las clases mientras el clúster corre sobre ellos, restricción operativa del laboratorio. Compromete la calidad del servicio de inferencia, en tanto la carga docente puede competir por los mismos recursos de cómputo que utiliza la plataforma. La mitigación consiste en que el esquema de cuotas y gobernanza de recursos, formulado en el objetivo específico 2 (Sección 3.2), reserve capacidad para las cargas del proyecto sin bloquear el uso académico habitual de los equipos, y en ejecutar fuera del horario de clases toda prueba que pueda degradar el rendimiento de las máquinas.

**Cambios de requisitos que obliguen a rehacer la infraestructura** El tutor advirtió que, al terminar el proyecto, pueden surgir necesidades nuevas que obliguen a reemplazar buena parte de lo construido. Compromete el alcance y el cronograma de mantenimiento posterior al proyecto. La mitigación es el compromiso adquirido por el equipo con GitOps, documentación y automatización de la infraestructura, de modo que un cambio de requisitos permita redesplegar o adaptar el clúster (por ejemplo, en otra sala del laboratorio) sin reconstruirlo desde cero.

**Dependencia de la arquitectura de aceleración de NVIDIA** Compromete el alcance de la solución fuera del escenario para el que se construye. Todos los nodos de trabajo que el laboratorio destina al clúster son estaciones equipadas con unidades de procesamiento gráfico de NVIDIA, y sobre esa arquitectura se apoyan tanto el aprovisionamiento declarativo de los aceleradores como la captura de las métricas que residen en el *firmware* de la tarjeta. La plataforma se diseña, por tanto, únicamente para nodos con aceleradores de ese fabricante, y su despliegue sobre equipos con aceleradores de otro proveedor exigiría sustituir las capas de aprovisionamiento y de telemetría. La mitigación consiste en mantener esa dependencia confinada a esas dos capas, de modo que el resto de la plataforma, que gobierna cuotas, reservas e identidad, opere sobre la abstracción de recurso que expone el orquestador y no sobre la tarjeta concreta.

**Límites de ancho de banda de la sala** Compromete la calidad de las pruebas de carga, en tanto la red de la sala del IAsLab tiene una capacidad de 10 Gbit/s. La configuración actual sirve cada modelo en un solo nodo, de modo que el proyecto no requiere una capacidad mayor para operar, y solo la necesitaría si la plataforma habilitara el reparto de un modelo entre varios nodos, modalidad prevista para cuando la infraestructura lo permita. La mitigación consiste en diseñar los escenarios de prueba de carga a partir de esa capacidad real, en lugar de extrapolar cifras de referentes de la literatura obtenidos en infraestructuras de mayor escala.